```
import {
  _decorator, BlockInputEvents, Color, Component, EventTouch, Graphics,
  JsonAsset, Label, Layers, Node, ResolutionPolicy, resources, sys, tween,
  UITransform, view, game as engineGame, Game, Sprite, SpriteFrame, Vec3, Rect, Size,
} from 'cc';
import { Action, adaptLevel, createGame, GameState, LevelDefinition, reduceGame, startLevel,
recoverInterruptedGame, PROP_PRICES, canClaimDoubleScore } from './core';
import { completeLevel, emptyProgress, ProgressSave, unlockedThrough } from
'./application/Progress';
import { LocalSave } from './platform/LocalSave';
import { RewardedVideo } from './platform/RewardedVideo';
import { theme } from './theme/Theme';
import { boardLayout } from './render/BoardLayout';
import { Feedback } from './platform/Feedback';
import { BrowserTestBridge } from './platform/BrowserTestBridge';
import { runtimeConfig } from './application/RuntimeConfig';
import { LoadingScreen } from './render/LoadingScreen';
import { TutorialScreen } from './render/TutorialScreen';
import { WinPanel } from './render/WinPanel';
const { ccclass } = _decorator;
type Prop = 'locate' | 'peek' | 'addTime';
const C = theme.colors;
/** One scene, engine-native UI, and a pure state reducer: no DOM / web framework. */
@ccclass('ShiseBoot')
export class ShiseBoot extends Component {
  private root!: Node;
  private height = 844;
  private top = 24;
  private bottom = 20;
  private levels: LevelDefinition[] = [];
  private save: ProgressSave = emptyProgress('');
  private storage = new LocalSave();
  private video = new RewardedVideo({ adUnitId: runtimeConfig.rewardedAdUnitId,
    runtime: (globalThis as unknown as { tt?: object }).tt || {} });
  private feedback!: Feedback;
  private bridge!: BrowserTestBridge;
  private homeFrame: SpriteFrame | null = null;
  private splashFrame: SpriteFrame | null = null;
  private iconsFrame: SpriteFrame | null = null;
  private homeArtFrame: SpriteFrame | null = null;
  private winFrame: SpriteFrame | null = null;
  private iconFrames = new Map<string, SpriteFrame>();
  private loading: LoadingScreen | null = null;
  private tutorial: TutorialScreen | null = null;
  private winPanel: WinPanel | null = null;
  private ready = false;
  private screen: 'home' | 'levels' | 'game' | 'shop' | 'tutorial' = 'home';
  private chapterPage = 0;
  private selectedCell: string | null = null;
  private selectedPool: number | null = null;
  private trayPage = 0;
  private modalOpen = false;
```

```
  private notice = '';
  private actionBusy = false;
  private idSequence = 0;
  private dismissedFailure: string | null = null;
  private timerLabel: Label | null = null;
  private clockAt = Date.now();
  private clockSavedAt = 0;
  private appHidden = false;
  onLoad(): void {
    const frame = view.getFrameSize();
    view.setDesignResolutionSize(390, 844, frame.width / frame.height > 0.65
      ? ResolutionPolicy.SHOW_ALL : ResolutionPolicy.FIXED_WIDTH);
    this.height = view.getVisibleSize().height;
    const safe = sys.getSafeAreaRect(false);
    this.top = Math.max(24, this.height - safe.yMax + 12);
    this.bottom = Math.max(20, safe.yMin + 12);
    try {
      const tt = (globalThis as unknown as { tt?: {
        getMenuButtonBoundingClientRect?: () => { bottom: number };
        getSystemInfoSync?: () => { windowWidth: number };
      } }).tt;
      const menu = tt?.getMenuButtonBoundingClientRect?.();
      const width = tt?.getSystemInfoSync?.().windowWidth;
      if (menu && width) this.top = Math.max(this.top, menu.bottom * 390 / width + 10);
    } catch { /* The safe-area fallback remains valid on hosts without a capsule API. */ }
    this.feedback = new Feedback(this.node);
    this.bridge = new BrowserTestBridge(runtimeConfig.browserAccessibility, () => ({
      screen: this.loading ? 'loading' : this.screen, modalOpen: this.modalOpen, version:
runtimeConfig.version,
      level: this.save.game?.level.number ?? null, status: this.save.game?.attempt.status ?? null,
      remainingTimeMs: this.save.game?.attempt.remainingTimeMs ?? null, points:
this.save.game?.wallet.points ?? 0, trayPage: this.trayPage + 1,
      inventory: this.save.game?.wallet.inventory ?? null, completed: this.save.completed.length,
    }), this.height);
    this.root = new Node('ShiseUI');
    this.root.layer = Layers.Enum.UI_2D;
    this.root.addComponent(UITransform).setContentSize(390, this.height);
    this.node.addChild(this.root);
    engineGame.on(Game.EVENT_HIDE, this.suspend, this);
    engineGame.on(Game.EVENT_SHOW, this.resume, this);
    this.renderLoading();
    // First launch always reaches the home screen. Tutorial is an explicit home entry.
    let assetsDone = 0;
    const art = ['splash-v2', 'home-v2', 'icons-v2', 'win-rosette-v1'];
    const artworkReady = Promise.all(art.map((name, index) => new Promise<void>(resolve => {
      resources.load(`art/${name}/spriteFrame`, SpriteFrame, (error, frame) => {
        if (!error && frame) {
          if (index === 0) { this.splashFrame = frame; this.loading?.setArtwork(frame); }
          if (index === 1) this.homeFrame = frame;
          if (index === 2) this.iconsFrame = frame;
          if (index === 3) this.winFrame = frame;
        } else console.error(`[Shise] Artwork unavailable: ${name}`, error);
```

```
        this.loading?.setProgress(0.12 + ++assetsDone * 0.20);
        resolve();
      });
    }));
    const loadingStarted = Date.now();
    resources.load('generated/levels', JsonAsset, async (error, asset) => {
      if (error || !asset) {
        this.clear();
        this.text('颜色还没准备好', 195, this.height / 2 - 24, 280, 30, 22);
        this.text('请检查 generated/levels 资源', 195, this.height / 2 + 16, 300, 40, 14,
C.textSecondary);
        return;
      }
      try {
        const content = asset.json as { contentVersion: string; levels: unknown[] };
        this.levels = content.levels.map(adaptLevel);
        this.save = this.storage.load(content.contentVersion, this.levels);
        if (!this.save.game) this.save.game = createGame(this.levels[0], this.nextId('attempt'));
        this.persist();
        // Only valid level versions can be resumed; do not reinterpret old boards.
        if (this.save.game && !this.levels.some(l => l.id === this.save.game!.level.id))
this.save.game = null;
        await artworkReady;
        this.loading?.setProgress(1, '准备好了');
        // Keep the real completion state visible long enough for readable launch notices.
        const finish = () => {
          if (this.loading?.isAgeInfoOpen) { this.scheduleOnce(finish, 0.2); return; }
          this.ready = true; this.renderHome();
        };
        this.scheduleOnce(finish, Math.max(0.25, (3000 - (Date.now() - loadingStarted)) / 1000));
      } catch (error) {
        console.error('[Shise] Failed to adapt level content', error);
        this.clear();
        this.text('题库版本需要更新', 195, this.height / 2, 300, 40, 22);
      }
    });
  }
  onDestroy(): void {
    this.tutorial?.dispose(); this.loading?.destroy(); this.winPanel?.dispose();
    engineGame.off(Game.EVENT_HIDE, this.suspend, this);
    engineGame.off(Game.EVENT_SHOW, this.resume, this);
    this.video.dispose();
  }
  private resume(): void { this.appHidden = false; this.clockAt = Date.now(); if (this.screen ===
'game') {
    this.renderGame();
  } }
  private suspend(): void {
    this.consumeTime(true); this.appHidden = true;
    this.selectedCell = null; this.selectedPool = null;
    if (this.save.game?.attempt.status === 'PreviewingReference')
      this.save.game = recoverInterruptedGame(this.save.game);
    this.persist();
```

```
  }
  private nextId(prefix: string): string { return `${prefix}-${Date.now()}-${++this.idSequence}`; }
  update(): void {
    const wasPlaying = this.save.game?.attempt.status === 'Playing';
    this.consumeTime();
    if (wasPlaying && this.save.game?.attempt.status === 'Failed' && this.screen === 'game')
this.renderGame();
  }
  private consumeTime(force = false): void {
    const now = Date.now();
    if (!this.ready || this.screen !== 'game' || this.modalOpen || this.appHidden ||
this.save.game?.attempt.status !== 'Playing') {
      this.clockAt = now; return;
    }
    const deltaMs = Math.max(0, now - this.clockAt);
    if (!force && deltaMs < 200) return;
    this.clockAt = now;
    if (!deltaMs) return;
    this.save.game = reduceGame(this.save.game, { type: 'elapse', deltaMs }).state;
    this.updateTimerLabel();
    if (now - this.clockSavedAt >= 5000 || this.save.game.attempt.status === 'Failed') {
      this.persist(); this.clockSavedAt = now;
    }
  }
  private updateTimerLabel(): void {
    if (!this.timerLabel?.isValid || !this.save.game) return;
    const seconds = Math.ceil(this.save.game.attempt.remainingTimeMs / 1000);
    this.timerLabel.string = `${Math.floor(seconds / 60)}:${String(seconds % 60).padStart(2, '0')}`;
    this.timerLabel.color = new Color().fromHEX(seconds <= 10 ? C.warning : C.text);
  }
  private persist(): boolean {
    const stored = this.storage.write(this.save);
    if (!stored || this.storage.error) this.notice = this.storage.error;
    return stored;
  }
  private persistConfirmedReward(retry = 0): void {
    if (this.persist()) return;
    this.notice = '奖励已确认，正在重试保存，请暂时不要关闭游戏';
    if (retry < 3) this.scheduleOnce(() => this.persistConfirmedReward(retry + 1), 2);
    else this.notice = '奖励已确认，但本地保存仍失败，请保留游戏并稍后重试';
  }
  private clear(): void {
    this.timerLabel = null;
    this.tutorial?.dispose(); this.tutorial = null;
    this.winPanel?.dispose(); this.winPanel = null;
    this.loading?.destroy(); this.loading = null;
    this.bridge.clear();
    for (const node of [...this.root.children]) { node.removeFromParent(); node.destroy(); }
    this.modalOpen = false;
    this.round(195, this.height / 2, 392, this.height + 2, 0, C.background);
  }
  private element(name: string, x: number, y: number, width: number, height: number, parent =
this.root): Node {
```

```
    const node = new Node(name);
    node.layer = Layers.Enum.UI_2D;
    node.addComponent(UITransform).setContentSize(width, height);
    node.setPosition(x - 195, this.height / 2 - y);
    parent.addChild(node);
    return node;
  }
  private round(x: number, y: number, width: number, height: number, radius: number,
    hex: string, parent = this.root, stroke?: string): Node {
    const node = this.element('Shape', x, y, width, height, parent);
    const g = node.addComponent(Graphics);
    g.fillColor = new Color().fromHEX(hex);
    g.roundRect(-width / 2, -height / 2, width, height, Math.min(radius, height / 2, width / 2));
    g.fill();
    if (stroke) { g.strokeColor = new Color().fromHEX(stroke); g.lineWidth = 2; g.stroke(); }
    return node;
  }
  private text(value: string, x: number, y: number, width: number, height: number,
    size = 16, color: string = C.text, bold = false, parent = this.root): Node {
    this.bridge.text(value);
    const node = this.element('Text', x, y, width, height, parent);
    const label = node.addComponent(Label);
    label.string = value;
    label.fontFamily = theme.typography.family;
    label.useSystemFont = true;
    label.fontSize = size;
    label.lineHeight = size * 1.45;
    label.color = new Color().fromHEX(color);
    label.isBold = bold;
    label.horizontalAlign = Label.HorizontalAlign.CENTER;
    label.verticalAlign = Label.VerticalAlign.CENTER;
    label.overflow = Label.Overflow.SHRINK;
    return node;
  }
  private button(value: string, x: number, y: number, width: number, height: number,
    onTap: () => void, fill: string = C.primary, color: string = C.onPrimary,
    parent = this.root, modalButton = false, semanticLabel?: string): Node {
    const node = this.round(x, y, width, height, theme.radii.button, fill, parent);
    // Label and shape share the same center; labels deliberately do not carry input.
    const label = new Node('Label');
    label.layer = Layers.Enum.UI_2D;
    label.addComponent(UITransform).setContentSize(width - 16, height);
    const component = label.addComponent(Label);
    component.string = value;
    component.fontFamily = theme.typography.family;
    component.useSystemFont = true;
    component.fontSize = height < 50 ? 15 : 18;
    component.lineHeight = 26;
    component.isBold = false;
    component.color = new Color().fromHEX(color);
```

```
    component.horizontalAlign = Label.HorizontalAlign.CENTER;
    component.verticalAlign = Label.VerticalAlign.CENTER;
    component.overflow = Label.Overflow.SHRINK;
    node.addChild(label);
    node.on(Node.EventType.TOUCH_START, () => {
      if ((!this.modalOpen || modalButton) && !this.save.settings.reducedMotion)
        tween(node).to(0.09, { scale: new Vec3(0.97, 0.97, 1) }).start();
    });
    const release = () => {
      if (this.save.settings.reducedMotion) node.setScale(1, 1, 1);
      else tween(node).to(0.09, { scale: new Vec3(1, 1, 1) }).start();
    };
    const invoke = () => { if ((!this.modalOpen || modalButton) && !this.actionBusy) onTap(); };
    this.bridge.control(semanticLabel || value, invoke, { x, y, width, height });
    node.on(Node.EventType.TOUCH_CANCEL, release);
    node.on(Node.EventType.TOUCH_END, (event: EventTouch) => {
      event.propagationStopped = true;
      release();
      invoke();
    });
    return node;
  }
  private renderLoading(): void {
    this.clear();
    this.loading = new LoadingScreen({ parent: this.root, width: 390, height: this.height,
      safeTop: this.top, safeBottom: this.bottom, artwork: this.splashFrame || undefined,
      registerControl: (label, invoke, bounds) => this.bridge.control(label, invoke, bounds),
      clearControls: labels => this.bridge.removeControls(labels) });
    this.loading.setProgress(0.05, '正在准备颜色');
  }
  private artwork(frame: SpriteFrame, x: number, y: number, width: number, height: number, parent =
this.root): Node {
    const node = this.element('Illustration', x, y, width, height, parent);
    const sprite = node.addComponent(Sprite); sprite.sizeMode = Sprite.SizeMode.CUSTOM;
sprite.spriteFrame = frame;
    node.getComponent(UITransform)!.setContentSize(width, height);
    return node;
  }
  private icon(name: string, x: number, y: number, size = 40, parent = this.root): void {
    const names = ['home', 'tutorial', 'settings', 'shop', 'locate', 'peek', 'addTime', 'undo',
'back', 'next', 'return', 'cancel'];
    const index = names.indexOf(name);
    if (this.iconsFrame && index >= 0) {
      let frame = this.iconFrames.get(name);
      if (!frame) {
        const source = this.iconsFrame;
        const width = source.originalSize.width / 4, height = source.originalSize.height / 3;
        frame = new SpriteFrame(); frame.texture = source.texture;
        frame.rect = new Rect(index % 4 * width, Math.floor(index / 4) * height, width, height);
        this.iconFrames.set(name, frame);
      }
      this.artwork(frame, x, y, size, size, parent);
    } else {
```

```
      const symbols: Record<string, string> = { home: '⌂', tutorial: '▤', settings: '⚙',
collection: '▧',
        locate: '✦', peek: '◉', addTime: '+', undo: '↶', back: '‹', next: '›', return: '↧',
cancel: '×' };
      this.text(symbols[name] || '·', x, y, size, size, size * 0.65, C.text, false, parent);
    }
  }
  private iconButton(name: string, label: string, x: number, y: number, size: number,
    invoke: () => void, fill: string = C.surface): void {
    this.button('', x, y, size, size, invoke, fill, C.text, this.root, false, label);
    this.icon(name, x, y, size * 0.85);
  }
  private openTutorial(): void {
    this.clear(); this.screen = 'tutorial';
    this.tutorial = new TutorialScreen(this.root, this.height, this.top, this.bottom,
      () => this.renderHome(), { onStart: () => {
          if (this.save.game && this.save.game.attempt.status !== 'Won') this.renderGame();
          else this.enterLevel(unlockedThrough(this.save, this.levels.map(l => l.id)));
        },
        registerControl: (label, invoke, bounds) => this.bridge.control(label, invoke, bounds),
        clearControls: () => this.bridge.clear(), onText: value => this.bridge.text(value),
reducedMotion: this.save.settings.reducedMotion });
  }
  private renderHome(): void {
    this.consumeTime(true);
    if (this.screen === 'game') this.persist();
    if (this.save.game?.attempt.status === 'PreviewingReference') {
      this.save.game = reduceGame(this.save.game, { type: 'endPeek' }).state; this.persist();
    }
    this.screen = 'home'; this.clear();
    const mainY = this.height - this.bottom - 206;
    if (this.homeFrame) {
      if (!this.homeArtFrame) {
        const source = this.homeFrame;
        const width = source.originalSize.width, height = source.originalSize.height;
        this.homeArtFrame = new SpriteFrame(); this.homeArtFrame.texture = source.texture;
        this.homeArtFrame.rect = new Rect(0, height * 0.18, width, height * 0.55);
        this.homeArtFrame.originalSize = new Size(width, height * 0.55);
      }
      const artTop = this.top + 144, artBottom = mainY - 49;
      const size = this.homeArtFrame.originalSize;
      const scale = Math.min(374 / size.width, Math.max(100, artBottom - artTop) / size.height);
      this.artwork(this.homeArtFrame, 195, (artTop + artBottom) / 2, size.width * scale, size.height
* scale);
    }
    this.round(83, this.top + 21, 122, 36, 18, C.butter);
    this.text(`✦ ${this.save.game?.wallet.points || 0}`, 83, this.top + 21, 116, 30, 14, C.text,
true);
    this.iconButton('settings', '设置', 344, this.top + 21, 44, () => this.openSettings());
    this.text('拾色之间', 195, this.top + 69, 304, 62, 38, C.text, true);
    this.text('100 道自然色彩 · 5 段拾色旅程', 195, this.top + 113, 304, 26, 12, C.textSecondary);
    const next = unlockedThrough(this.save, this.levels.map(l => l.id));
    const resume = this.save.game;
    const continuing = resume && resume.attempt.status !== 'Won' && (resume.attempt.elapsedMs > 0 ||
resume.attempt.movesSpent > 0);
    const levelNumber = continuing ? resume.level.number : next + 1;
```

```
    this.round(195, mainY + 5, 310, 62, 28, '#E7B5A2');
    this.button(continuing ? '继续游戏' : '开始游戏', 195, mainY, 310, 62,
      () => continuing ? this.renderGame() : this.enterLevel(next), C.primary, '#FFFFFF');
    this.text(`第 ${levelNumber} 关 · ${this.levels[levelNumber - 1]?.title || '自然的颜色'}`,
      195, mainY + 49, 320, 24, 12, C.textSecondary);
    const cards = [
      { icon: 'home', label: '关卡', fill: C.mint, action: () => { this.chapterPage =
Math.floor(next / 20); this.renderLevels(); } },
      { icon: 'tutorial', label: '教程', fill: C.butter, action: () => this.openTutorial() },
      { icon: 'addTime', label: '道具', fill: C.lilac, action: () => this.renderShop() },
    ];
    cards.forEach((card, index) => {
      const x = 83 + index * 112, y = mainY + 129;
      this.round(x, y + 3, 100, 96, 22, '#EDE5DA');
      this.button('', x, y, 100, 96, card.action, card.fill, C.text, this.root, false, card.label);
      this.icon(card.icon, x, y - 13, 66);
      this.text(card.label, x, y + 28, 88, 24, 14, C.text, true);
    });
    this.text(`${this.save.completed.length} / 100`, 195, this.height - this.bottom - 9, 200, 18,
12, C.textSecondary);
  }
  private renderShop(): void {
    this.consumeTime(true); if (this.screen === 'game') this.persist(); this.screen = 'shop';
this.clear();
    this.iconButton('back', '返回首页', 40, this.top + 22, 44, () => this.renderHome());
    this.text('道具', 195, this.top + 20, 190, 36, 24, C.text, true);
    this.round(195, this.top + 80, 184, 44, 22, C.butter);
    this.text(`✦  ${this.save.game?.wallet.points || 0} 积分`, 195, this.top + 80, 180, 40, 17,
C.text, true);
    const descriptions = {
      locate: ['点睛', '归位并固定 1 块颜色', '每关最多 2 次'],
      peek: ['窥色', '展示一条完整色路 3 秒', '每关最多 1 次'],
      addTime: ['续时', '为本关增加 30 秒', '每关最多 2 次'],
    };
    (['locate', 'peek', 'addTime'] as Prop[]).forEach((prop, index) => {
      const y = this.top + 175 + index * 133;
      this.round(195, y + 4, 342, 122, 24, '#EBE4D8');
      this.round(195, y, 342, 122, 24, C.surface);
      this.round(79, y - 10, 76, 76, 22, [C.mint, C.lilac, C.butter][index]);
      this.icon(prop, 79, y - 10, 68);
      this.text(descriptions[prop][0], 223, y - 36, 200, 28, 19, C.text, true);
      this.text(descriptions[prop][1], 231, y - 8, 216, 24, 13, C.textSecondary);
      this.text(`库存 ${this.save.game?.wallet.inventory[prop] || 0} · ${descriptions[prop][2]}`,
        217, y + 15, 216, 22, 11, C.textSecondary);
      this.button(`${PROP_PRICES[prop]} 积分购买`, 235, y + 44, 196, 32,
        () => this.buyProp(prop), C.primary, '#FFFFFF', this.root, false, `购买${descriptions[prop]
[0]}`);
    });
    this.text(this.notice || '通关获得积分，最佳成绩提升还能补发差额', 195,
      this.height - this.bottom - 24, 344, 40, 12, C.textSecondary);
  }
  private buyProp(prop: Prop, inGame = false): void {
    if (!this.save.game) return;
    const before = this.save.game;
    const result = reduceGame(before, { type: 'purchaseProp', prop });
```

```
    if (result.changed) {
      this.save.game = result.state;
      if (!this.persist()) { this.save.game = before; this.notice = '保存未成功，积分未扣除，请稍后再试';
}
      else this.notice = '已放入道具背包';
    } else this.notice = result.reason || '积分不足，通关后再来看看';
    if (inGame) { this.renderGame(); this.openProp(prop); }
    else this.renderShop();
  }
  private openSettings(): void {
    const { layer, top } = this.sheet('设置', '', 414);
    const labels = { sound: '轻音效', haptics: '轻触反馈', reducedMotion: '减少动效' };
    (['sound', 'haptics', 'reducedMotion'] as const).forEach((key, i) => {
      this.button(`${labels[key]}   ${this.save.settings[key] ? '已开启' : '已关闭'}`,
        195, top + 157 + i * 57, 280, 48, () => {
          this.save.settings[key] = !this.save.settings[key]; this.persist();
          this.renderHome(); this.openSettings();
        }, this.save.settings[key] ? C.mint : C.disabled, C.text, layer, true);
    });
    this.button('好了，继续拾色', 195, top + 348, 280, 48, () => this.renderHome(), C.primary,
C.onPrimary, layer, true);
  }
  private renderLevels(): void {
    this.consumeTime(true);
    if (this.screen === 'game') this.persist();
    this.screen = 'levels';
    this.clear();
    this.button('‹', 40, this.top + 24, 48, 48, () => this.renderHome(), C.surface, C.text);
    const first = this.levels[this.chapterPage * 20];
    this.text(`第 ${this.chapterPage + 1} 章 · ${first.chapterName}`, 195, this.top + 26, 260, 40,
20, C.text, true);
    this.text('一关一关，拾起沿途的颜色', 195, this.top + 78, 320, 30, 15, C.textSecondary);
    const unlocked = unlockedThrough(this.save, this.levels.map(l => l.id));
    for (let i = 0; i < 20; i++) {
      const index = this.chapterPage * 20 + i;
      const level = this.levels[index];
      const allowed = index <= unlocked;
      const done = this.save.completed.includes(level.id);
      this.button(`${index + 1}${done ? ' ✓' : ''}`,
        72 + i % 4 * 82, this.top + 157 + Math.floor(i / 4) * 72, 62, 56,
        () => { if (allowed) this.enterLevel(index); else { this.notice = `完成第 ${index} 关后解锁`;
this.renderLevels(); } },
        done ? C.mint : allowed ? C.butter : C.disabled, allowed ? C.text : C.textSecondary);
    }
    const pagerY = Math.min(this.height - this.bottom - 46, this.top + 573);
    this.button('上一章', 100, pagerY, 120, 48, () => {
      if (this.chapterPage > 0) { this.chapterPage--; this.renderLevels(); }
    }, C.surface, C.text);
    this.button('下一章', 290, pagerY, 120, 48, () => {
      if (this.chapterPage < 4) { this.chapterPage++; this.renderLevels(); }
    }, C.surface, C.text);
    this.text(this.notice || '完成当前关卡，就能开启下一道风景', 195, this.height - this.bottom, 342, 28,
13, C.textSecondary);
  }
  private enterLevel(index: number): void {
```

```
    const allowed = unlockedThrough(this.save, this.levels.map(l => l.id));
    if (index < 0 || index >= this.levels.length || index > allowed) return;
    this.save.game = this.save.game
      ? startLevel(this.save.game, this.levels[index], this.nextId('attempt'))
      : createGame(this.levels[index], this.nextId('attempt'));
    this.selectedCell = null;
    this.selectedPool = null;
    this.notice = '';
    this.trayPage = 0;
    this.persist();
    this.renderGame();
  }
  private dispatch(action: Action): void {
    if (!this.save.game || this.actionBusy) return;
    this.consumeTime(true);
    const result = reduceGame(this.save.game, action);
    if (result.changed) {
      this.save.game = result.state;
      this.selectedCell = null; this.selectedPool = null;
      completeLevel(this.save, this.levels.map(l => l.id));
      if (['place', 'move', 'locate'].includes(action.type)) this.feedback.play(
        result.state.attempt.status === 'Won' ? 'complete' : 'place', this.save.settings.sound,
this.save.settings.haptics);
      this.notice = '';
      this.persist();
    } else if (result.reason) {
      this.notice = this.friendlyReason(result.reason);
    }
    this.renderGame();
  }
  private friendlyReason(reason: string): string {
    if (/stock|inventory/i.test(reason)) return '这个道具用完啦';
    if (/limit|maximum/i.test(reason)) return '这次的使用次数已用完';
    if (/lock|anchor/i.test(reason)) return '小圆点标记的颜色已经固定';
    return reason;
  }
  private renderGame(): void {
    this.consumeTime(true);
    if (!this.save.game) { this.renderHome(); return; }
    this.screen = 'game';
    this.clear();
    const state = this.save.game;
    const { level, attempt } = state;
    const layout = boardLayout(this.height, this.top, this.bottom, level.grid.columns,
      level.grid.rows, attempt.pool.length);
    const { cellSize, pitch, boardStartX, boardStartY } = layout;
    this.iconButton('back', '返回首页', 40, this.top + 22, 44, () => {
      this.selectedCell = null; this.selectedPool = null; this.renderHome();
    });
    this.iconButton('settings', '关卡菜单', 350, this.top + 22, 44, () => this.openGameMenu());
    this.text(`${level.title}`, 195, this.top + 14, 244, 30, 19, C.text, true);
```

```
    this.text(`第${level.number} 关`, 152, this.top + 46, 106, 22, 12, C.textSecondary);
    this.round(253, this.top + 47, 92, 30, 15, C.butter);
    this.timerLabel = this.text('', 253, this.top + 47, 88, 30, 17, C.text,
true).getComponent(Label);
    this.updateTimerLabel();
    const xy = (cellId: string) => {
      const cell = level.cells.find(c => c.id === cellId)!;
      return { x: boardStartX + cell.x * pitch, y: boardStartY + cell.y * pitch };
    };
    const groupWidth = Math.min(370, Math.max(264, layout.groupWidth + 30));
    this.round(195, layout.groupTop + layout.groupHeight / 2 + 5, groupWidth,
      layout.groupHeight + 36, 27, '#E8E3D7');
    this.round(195, layout.groupTop + layout.groupHeight / 2, groupWidth,
      layout.groupHeight + 36, 27, '#FFFEFB');
    this.round(195, layout.trayTop + layout.trayHeight / 2, Math.min(groupWidth - 12,
layout.trayWidth + 18),
      layout.trayHeight + 16, 16, '#F0F1E9');
    const tracks = this.element('Tracks', 195, 0, 390, this.height);
    tracks.setPosition(0, this.height / 2);
    const g = tracks.addComponent(Graphics);
    g.strokeColor = new Color().fromHEX(C.track);
    g.lineWidth = 2;
    level.paths.forEach(path => {
      path.cellIds.forEach((id, i) => {
        const p = xy(id);
        if (!i) g.moveTo(p.x - 195, -p.y); else g.lineTo(p.x - 195, -p.y);
      });
      g.stroke();
    });
    const tileHex = (id: string | null) => level.tiles.find(t => t.id === id)?.hex || C.surface;
    const previewCells = attempt.preview
      ? level.paths.find(path => path.id === attempt.preview!.pathId)?.cellIds ?? [] : [];
    for (const cell of level.cells) {
      const p = xy(cell.id);
      const tile = attempt.board[cell.id];
      const locked = level.anchors.includes(cell.id) || attempt.auxiliaryLocks.includes(cell.id);
      const hex = previewCells.includes(cell.id) ? cell.targetHex : tileHex(tile);
      if (this.selectedCell === cell.id) this.round(p.x, p.y, cellSize + 8, cellSize + 8, 12,
'#A6CEC3');
      const node = this.round(p.x, p.y, cellSize, cellSize, theme.radii.tile, hex,
        this.root, tile === null && !previewCells.includes(cell.id) ? '#DDD7CB' : undefined);
      node.on(Node.EventType.TOUCH_END, (event: EventTouch) => {
        event.propagationStopped = true;
        if (!this.modalOpen) this.tapCell(cell.id);
      });
      if (!locked) this.bridge.control(`上方${tile ? '色块' : '空槽'} ${cell.id}`, () => { if (!
this.modalOpen) this.tapCell(cell.id); }, { x: p.x, y: p.y, width: cellSize, height: cellSize });
      if (locked) {
        this.round(p.x + cellSize / 2 + 1, p.y - cellSize / 2 - 1, 7, 7, 4, C.surface);
        const marker = this.round(p.x + cellSize / 2 + 1, p.y - cellSize / 2 - 1,
          4, 4, level.anchors.includes(cell.id) ? 2 : 0, C.text);
        if (!level.anchors.includes(cell.id)) marker.setRotationFromEuler(0, 0, 45);
      }
    }
```

```
    // Instructions live in the interactive tutorial, not permanently on the play board.
    if (attempt.preview || this.notice) this.text(attempt.preview ? '参考色路 · 3 秒' : this.notice,
      195, layout.trayLabelY - 6, 352, 26, 12, C.textSecondary);
    const pages = Math.max(1, Math.ceil(attempt.pool.length / layout.slotsPerPage));
    this.trayPage = Math.min(pages - 1, this.trayPage);
    const firstSlot = this.trayPage * layout.slotsPerPage;
    const lastSlot = Math.min(attempt.pool.length, firstSlot + layout.slotsPerPage);
    for (let i = firstSlot; i < lastSlot; i++) {
      const local = i - firstSlot;
      const id = attempt.pool[i];
      const x = 195 - (layout.trayColumns - 1) * layout.trayPitch / 2
        + local % layout.trayColumns * layout.trayPitch;
      const y = layout.trayTop + 22 + Math.floor(local / layout.trayColumns) * layout.trayPitch;
      if (this.selectedPool === i) this.round(x, y, 52, 52, 12, '#A6CEC3');
      const node = this.round(x, y, 44, 44, 8, tileHex(id), this.root, id ? undefined : C.border);
      node.on(Node.EventType.TOUCH_END, (event: EventTouch) => {
        event.propagationStopped = true;
        if (!this.modalOpen) this.tapPool(i);
      });
      this.bridge.control(`候选槽 ${i + 1}${id ? ' ' + tileHex(id) : ' 空'}`, () => { if (!
this.modalOpen) this.tapPool(i); }, { x, y, width: 44, height: 44 });
    }
    if (pages > 1) {
    this.iconButton('back', '上一页候选', 130, layout.pagerY, 40, () => {
      if (this.trayPage > 0) { this.trayPage--; this.renderGame(); }
    });
    this.text(`${this.trayPage + 1} / ${pages}`, 195, layout.pagerY, 70, 26, 12, C.textSecondary);
    this.iconButton('next', '下一页候选', 260, layout.pagerY, 40, () => {
      if (this.trayPage + 1 < pages) { this.trayPage++; this.renderGame(); }
    });
    }
    if (attempt.status === 'Failed') {
      this.button('继续本关', 64, layout.actionsY, 96, 48, () => this.openFail(), C.mint, C.text);
      this.button('免费重开', 195, layout.actionsY, 144, 48,
        () => this.dispatch({ type: 'restart', attemptId: this.nextId('attempt') }), C.surface,
C.text);
      this.button('选关', 326, layout.actionsY, 92, 48,
        () => { this.chapterPage = level.chapter - 1; this.renderLevels(); }, C.surface, C.text);
    } else {
    this.iconButton('cancel', '取消选择', 133, layout.actionsY, 44, () => {
      this.selectedCell = null; this.selectedPool = null; this.notice = ''; this.renderGame();
    }, this.selectedCell || this.selectedPool !== null ? C.surface : C.background);
    this.iconButton('return', '放回待选区', 195, layout.actionsY, 44, () => {
      if (this.selectedCell) this.dispatch({ type: 'returnToTray', fromCellId: this.selectedCell });
      else { this.notice = '先点选上方已填的颜色'; this.renderGame(); }
    }, this.selectedCell ? C.mint : C.background);
    this.iconButton('undo', '撤销', 257, layout.actionsY, 44,
      () => this.dispatch({ type: 'undo' }), C.surface);
    }
    (['locate', 'peek', 'addTime'] as Prop[]).forEach((prop, i) => {
      const names = { locate: '点睛', peek: '窥色', addTime: '续时' };
      const x = 79 + i * 116;
```

```
      this.round(x, layout.propsY + 3, 100, 70, 20, '#EAE3D8');
      this.button('', x, layout.propsY, 100, 70,
        () => this.openProp(prop), attempt.status === 'Playing' ? [C.mint, C.lilac, C.butter][i] :
C.disabled,
        C.text, this.root, false, names[prop]);
      this.icon(prop, x, layout.propsY - 10, 48);
      this.text(names[prop], x, layout.propsY + 22, 80, 22, 12, C.text);
      this.round(x + 33, layout.propsY - 23, 22, 22, 11, C.surface);
      this.text(String(state.wallet.inventory[prop]), x + 33, layout.propsY - 23, 22, 22, 11,
C.text);
    });
    if (attempt.status === 'Won') this.openWin();
    else if (attempt.status === 'Failed' && this.dismissedFailure !== attempt.id) this.openFail();
  }
  private tapPool(index: number): void {
    this.consumeTime(true);
    const state = this.save.game!;
    if (state.attempt.status !== 'Playing') { this.renderGame(); return; }
    if (state.attempt.pool[index]) {
      this.selectedPool = this.selectedPool === index ? null : index;
      this.selectedCell = null; this.notice = ''; this.renderGame();
    } else if (this.selectedCell) {
      this.dispatch({ type: 'returnToTray', fromCellId: this.selectedCell, toPoolIndex: index });
    }
  }
  private tapCell(cellId: string): void {
    this.consumeTime(true);
    const state = this.save.game!;
    if (state.attempt.status !== 'Playing') { this.renderGame(); return; }
    if (state.level.anchors.includes(cellId) || state.attempt.auxiliaryLocks.includes(cellId)) {
      this.notice = '带小圆点的颜色已经固定'; this.renderGame(); return;
    }
    if (this.selectedCell === cellId) {
      this.selectedCell = null; this.renderGame(); return;
    }
    if (state.attempt.board[cellId] !== null) {
      if (this.selectedPool !== null || this.selectedCell !== null) {
        this.notice = '这里已有颜色，请先取回或选一个空格';
      } else { this.selectedCell = cellId; this.notice = ''; }
      this.renderGame(); return;
    }
    if (this.selectedPool !== null) {
      this.dispatch({ type: 'place', poolIndex: this.selectedPool, toCellId: cellId });
    } else if (this.selectedCell) {
      this.dispatch({ type: 'move', fromCellId: this.selectedCell, toCellId: cellId });
    } else { this.notice = '先点选下方的一块颜色'; this.renderGame(); }
  }
  private sheet(title: string, subtitle: string, height = 360): { layer: Node; top: number; center:
number } {
    this.consumeTime(true);
    this.modalOpen = true; this.bridge.modal();
    const layer = this.element('Modal', 195, this.height / 2, 390, this.height);
    layer.setPosition(0, 0);
```

```
    const overlay = this.round(195, this.height / 2, 390, this.height, 0, C.scrim, layer);
    const graphics = overlay.getComponent(Graphics)!;
    const alpha = Math.round(255 * theme.controls.scrimOpacity);
    const tint = new Color().fromHEX(C.scrim);
    tint.a = alpha;
    graphics.clear(); graphics.fillColor = tint;
    graphics.rect(-195, -this.height / 2, 390, this.height); graphics.fill();
    overlay.addComponent(BlockInputEvents);
    const center = this.height / 2;
    const top = center - height / 2;
    const panel = this.round(195, center, 342, height, theme.radii.modal, C.surface, layer);
    panel.addComponent(BlockInputEvents);
    this.text(title, 195, top + 52, 286, 42, 25, C.text, true, layer);
    this.text(subtitle, 195, top + 99, 292, 52, 15, C.textSecondary, false, layer);
    return { layer, top, center };
  }
  private openGameMenu(): void {
    const state = this.save.game;
    if (!state || state.attempt.status === 'AdPending') return;
    const { layer, top } = this.sheet('这一道颜色，慢慢来', '离开会保留当前盘面', 360);
    this.button('继续这一关', 195, top + 157, 280, 48, () => this.renderGame(), C.mint, C.text,
layer, true);
    this.button('免费重新开始', 195, top + 220, 280, 48, () => {
      const confirm = this.sheet('重新拾起这些颜色？', '盘面恢复开局；已使用道具不会返还', 290);
      this.button('确定重开', 195, confirm.top + 162, 280, 48,
        () => this.dispatch({ type: 'restart', attemptId: this.nextId('attempt') }), C.primary,
C.onPrimary, confirm.layer, true);
      this.button('再想想', 195, confirm.top + 224, 280, 44,
        () => this.renderGame(), C.mint, C.text, confirm.layer, true);
    }, C.surface, C.text, layer, true);
    this.button('返回首页', 195, top + 283, 280, 48,
      () => this.renderHome(), C.surface, C.textSecondary, layer, true);
  }
  private openWin(): void {
    const state = this.save.game!;
    const attempt = state.attempt;
    this.modalOpen = true; this.bridge.modal();
    this.winPanel?.dispose();
    this.winPanel = new WinPanel({
      parent: this.root, width: 390, height: this.height, safeTop: this.top, safeBottom:
this.bottom,
      score: attempt.score || 0, pointsEarned: attempt.pointsEarned, elapsedMs: attempt.elapsedMs,
      movesSpent: attempt.movesSpent, levelNumber: state.level.number, levelTitle:
state.level.title,
      doubleAvailable: canClaimDoubleScore(state), artwork: this.winFrame,
      onDouble: () => this.requestVideo('doubleScore'),
      onNext: () => state.level.number < 100 ? this.enterLevel(state.level.number) :
this.renderLevels(),
      onHome: () => this.renderHome(),
      registerControl: (label, invoke, bounds) => this.bridge.control(label, invoke, bounds),
      onText: value => this.bridge.text(value),
    });
  }
  private openFail(): void {
    const state = this.save.game!;
```

```
    const max = state.level.challenge.revive.maxPerAttempt;
    const canRevive = state.attempt.revivesUsed < max;
    const entitlement = state.wallet.pendingRevives[0];
    const { layer, top } = this.sheet('时间到了', '当前盘面已保留，复活后接着完成', 414);
    this.button('×', 327, top + 24, 40, 40, () => {
      this.dismissedFailure = state.attempt.id; this.renderGame();
    }, C.surface, C.text, layer, true);
    this.text(`本关还可复活 ${Math.max(0, max - state.attempt.revivesUsed)} 次`, 195, top + 142,
      290, 26, 13, C.textSecondary, false, layer);
    if (canRevive) {
      this.button(entitlement ? `使用已获得的 +${entitlement.amount} 秒` : `▷ 看视频复活 +${state.level.challenge.revive.addSeconds} 秒`,
        195, top + 196, 288, 56, () => {
          this.dismissedFailure = null;
          if (entitlement) this.dispatch({ type: 'applyPendingRevive', requestId: entitlement.requestId });
          else this.requestVideo('revive');
        }, C.primary, '#FFFFFF', layer, true);
    } else this.text('本关复活次数已用完', 195, top + 196, 288, 56, 16, C.textSecondary, false, layer);
    this.button('免费重新开始', 195, top + 268, 288, 48,
      () => this.dispatch({ type: 'restart', attemptId: this.nextId('attempt') }), C.mint, C.text, layer, true);
    this.button('返回关卡', 195, top + 335, 220, 48,
      () => { this.chapterPage = state.level.chapter - 1; this.renderLevels(); }, C.surface, C.textSecondary, layer, true);
  }
  private openProp(prop: Prop): void {
    this.consumeTime(true);
    const state = this.save.game!;
    if (state.attempt.status !== 'Playing') { this.renderGame(); return; }
    const names = { locate: '点睛', peek: '窥色', addTime: '续时' };
    const descriptions = { locate: '归位并固定 1 块颜色', peek: '选择一条色路，展示完整颜色 3 秒', addTime: '为本关倒计时增加 30 秒' };
    const { layer, top } = this.sheet(names[prop], descriptions[prop], 430);
    const limits = { locate: 2, peek: 1, addTime: 2 };
    const usesLeft = limits[prop] - state.attempt.propUses[prop];
    const stock = state.wallet.inventory[prop];
    this.text(`库存 ${stock} · 本关还可用 ${usesLeft} 次`, 195, top + 142, 290, 28, 14, C.textSecondary, false, layer);
    this.text(this.notice || `积分余额 ${state.wallet.points}`, 195, top + 173, 290, 26, 13, C.textSecondary, false, layer);
    if (usesLeft <= 0) this.text('本关使用次数已用完', 195, top + 225, 280, 48, 16, C.textSecondary, false, layer);
    else this.button(stock > 0 ? '使用道具' : `${PROP_PRICES[prop]} 积分购买 1 个`, 195, top + 225, 280, 50, () => {
      if (stock <= 0) { this.buyProp(prop, true); return; }
      this.notice = '';
      if (prop === 'peek') { this.openPathPicker(); return; }
      this.dispatch(prop === 'locate' ? { type: 'locate' } : { type: 'addTime' });
    }, C.primary, '#FFFFFF', layer, true);
    if (state.attempt.propRestockAdsGranted < 1 && stock === 0 && usesLeft > 0)
      this.button('▷ 看视频领取 1 个', 195, top + 293, 280, 48,
        () => this.requestVideo('restock', prop), C.butter, C.text, layer, true);
    else this.button(`积分购买 · ${PROP_PRICES[prop]} 分`, 195, top + 293, 280, 48,
      () => this.buyProp(prop, true), C.lilac, C.text, layer, true);
    this.button('继续游戏', 195, top + 365, 230, 44,
      () => { this.notice = ''; this.renderGame(); }, C.surface, C.text, layer, true);
  }
  private openPathPicker(): void {
```

```
    this.renderGame();
    const paths = this.save.game!.level.paths;
    const height = Math.min(this.height - this.top - this.bottom - 24, 180 +
Math.ceil(paths.length / 3) * 50);
    const { layer, top } = this.sheet('想看看哪条色路？', '参考 3 秒后自动收起', height);
    this.button('×', 327, top + 24, 40, 40, () => this.renderGame(), C.surface, C.text, layer,
true);
    paths.forEach((path, i) => this.button(`色路 ${i + 1}`, 89 + i % 3 * 106, top + 153 +
Math.floor(i / 3) * 50,
      96, 44, () => {
        this.dispatch({ type: 'peek', pathId: path.id, nowMs: Date.now() });
        const attemptId = this.save.game!.attempt.id;
        this.scheduleOnce(() => {
          if (this.save.game?.attempt.id !== attemptId) return;
          const result = reduceGame(this.save.game, { type: 'expirePeek', nowMs: Date.now() });
          if (result.changed) {
            this.save.game = result.state; this.persist();
            if (this.screen === 'game') this.renderGame();
          }
        }, 3);
      }, C.lilac, C.text, layer, true));
  }
  private requestVideo(rewardType: 'revive' | 'restock' | 'doubleScore', prop?: Prop): void {
    if (!this.video.available) {
      this.renderGame();
      const { layer, top } = this.sheet('广告暂不可用', '激励视频广告位尚未开通，开通后即可领取奖励', 280);
      this.button('返回游戏', 195, top + 197, 260, 48, () => this.renderGame(), C.mint, C.text,
layer, true);
      return;
    }
    const requestId = this.nextId('reward');
    const before = this.save.game!;
    const result = reduceGame(before, { type: 'beginReward', requestId, rewardType, prop });
    if (!result.changed) { this.notice = result.reason || '本次奖励暂不可领取'; this.renderGame();
return; }
    this.save.game = result.state;
    if (!this.persist()) { this.save.game = before; this.renderGame(); return; }
    this.renderGame();
    this.sheet('正在准备视频', '完整观看后发放奖励；提前关闭不会扣除道具或积分', 240);
    this.video.show(outcome => {
      if (!this.save.game) return;
      const settled = reduceGame(this.save.game, { type: 'settleReward', requestId, outcome });
      this.save.game = settled.state;
      this.notice = outcome === 'completed' ? '' : outcome === 'failed'
        ? '视频暂不可用，盘面和库存已保留' : '未完整观看，本次没有领取奖励';
      if (outcome === 'completed') this.persistConfirmedReward();
      else this.persist();
      if (this.screen === 'game') this.renderGame();
      else if (this.screen === 'shop') this.renderShop();
      else this.renderHome();
    });
  }
}
/** Pure game rules. No Cocos, platform SDK, clock, randomness or storage dependencies. */
export const RULES_VERSION = 'click-fill-timed-flex-v3' as const;
```

```
export type Prop = 'locate' | 'peek' | 'addTime';
export type Inventory = Record<Prop, number>;
export type Board = Record<string, string | null>;
export type AttemptStatus = 'Playing' | 'Won' | 'Failed' | 'AdPending' | 'PreviewingReference';
export interface LevelDefinition {
  gameplayMode: 'click-fill';
  inputMode: 'tap';
  id: string;
  number: number;
  title: string;
  chapter: number;
  chapterName: string;
  contentVersion: string;
  /** Different puzzle designs only compare performance within the same scoring epoch. */
  scoreEpoch?: string;
  grid: { columns: number; rows: number };
  cells: { id: string; x: number; y: number; targetHex: string }[];
  paths: { id: string; cellIds: string[] }[];
  anchors: string[];
  tiles: { id: string; hex: string }[];
  initialBoard: Board;
  initialPool: string[];
  /** Authored initial slot indexes; movable tiles may occupy any empty tray slot at runtime. */
  trayHomeByTile: Record<string, number>;
  tutorialText: string;
  challenge: { mode: 'time'; timeLimitSeconds: number; revive: { maxPerAttempt: number; addSeconds:
number } };
}
export interface UndoEntry {
  kind: 'place' | 'move' | 'returnToTray' | 'locate';
  board: Board;
  pool: (string | null)[];
  auxiliaryLocks: string[];
}
export interface Attempt {
  rulesVersion: typeof RULES_VERSION;
  id: string;
  levelId: string;
  levelContentVersion: string;
  revision: number;
  status: AttemptStatus;
  board: Board;
  pool: (string | null)[];
  auxiliaryLocks: string[];
  initialTimeMs: number;
  remainingTimeMs: number;
  elapsedMs: number;
  movesSpent: number;
  totalTimeGrantedMs: number;
  score: number | null;
  pointsEarned: number;
```

```
  undoStack: UndoEntry[];
  propUses: Inventory;
  revivesUsed: number;
  propRestockAdsGranted: number;
  reviveAdsGranted: number;
  pendingRewardRequestId: string | null;
  previousStatusBeforeAd: 'Playing' | 'Failed' | 'Won' | null;
  preview: { pathId: string; expiresAtMs: number } | null;
}
export interface RewardRequest {
  id: string;
  attemptId: string;
  levelId: string;
  levelContentVersion: string;
  rewardType: 'revive' | 'restock' | 'doubleScore';
  /** Double-score qualification belongs to the puzzle design that created the request. */
  scoreEpoch?: string;
  amount: number;
  prop: Prop | null;
  status: 'pending' | 'granted' | 'cancelled' | 'failed';
}
export interface PendingRevive {
  requestId: string;
  originAttemptId: string;
  amount: number;
}
export interface LevelScore {
  scoreEpoch?: string;
  bestScore: number;
  doubleScoreClaimed: boolean;
}
export interface Wallet {
  points: number;
  levelScores: Record<string, LevelScore>;
  inventory: Inventory;
  rewardLedger: Record<string, RewardRequest>;
  pendingRevives: PendingRevive[];
}
export interface GameState {
  level: LevelDefinition;
  attempt: Attempt;
  wallet: Wallet;
}
export type Action =
  | { type: 'place'; poolIndex: number; toCellId: string }
  | { type: 'move'; fromCellId: string; toCellId: string }
  | { type: 'returnToTray'; fromCellId: string; toPoolIndex?: number }
  | { type: 'undo' }
  | { type: 'locate' }
  | { type: 'peek'; pathId: string; nowMs: number }
```

```
  | { type: 'endPeek' }
  | { type: 'expirePeek'; nowMs: number }
  | { type: 'suspend' }
  | { type: 'addTime' }
  | { type: 'elapse'; deltaMs: number }
  | { type: 'purchaseProp'; prop: Prop }
  | { type: 'restart'; attemptId: string }
  | { type: 'beginReward'; requestId: string; rewardType: 'revive' | 'restock' | 'doubleScore';
prop?: Prop }
  /** The platform adapter must authenticate full viewing; this command is not an ad SDK. */
  | { type: 'settleReward'; requestId: string; outcome: 'completed' | 'cancelled' | 'failed' }
  | { type: 'applyPendingRevive'; requestId: string };
export type DomainEvent =
  | { type: 'won'; levelId: string; attemptId: string; score: number; pointsEarned: number }
  | { type: 'failed'; levelId: string; attemptId: string }
  | { type: 'rewardRequested'; requestId: string }
  | { type: 'rewardGranted'; requestId: string; deferred: boolean };
export interface Transition {
  state: GameState;
  changed: boolean;
  reason?: string;
  events: DomainEvent[];
}
export const DEFAULT_SCORE_EPOCH = 'base-v1';
export const PROP_LIMITS: Readonly<Inventory> = Object.freeze({ locate: 2, peek: 1, addTime: 2 });
export const PROP_PRICES: Readonly<Inventory> = Object.freeze({ locate: 120, peek: 80, addTime:
100 });
export const INITIAL_INVENTORY: Readonly<Inventory> = Object.freeze({ locate: 1, peek: 1, addTime: 1
});
export const STARTER_INVENTORY: Readonly<Inventory> = Object.freeze({ locate: 3, peek: 2, addTime: 1
});
const own = (object: object, key: string): boolean => Object.prototype.hasOwnProperty.call(object,
key);
const emptyInventory = (): Inventory => ({ locate: 0, peek: 0, addTime: 0 });
const hex = (value: unknown): string => {
  if (typeof value !== 'string' || !/^#[0-9a-f]{6}$/i.test(value)) throw new Error('Invalid HEX
colour');
  return value.toUpperCase();
};
const object = (value: unknown, name: string): Record<string, unknown> => {
  if (!value || typeof value !== 'object' || Array.isArray(value)) throw new Error(`Invalid ${name}
`);
  return value as Record<string, unknown>;
};
const array = (value: unknown, name: string): unknown[] => {
  if (!Array.isArray(value)) throw new Error(`Invalid ${name}`);
  return value;
};
const string = (value: unknown, name: string): string => {
  if (typeof value !== 'string' || value.length === 0) throw new Error(`Invalid ${name}`);
  return value;
};
const integer = (value: unknown, name: string, minimum = 0): number => {
  if (typeof value !== 'number' || !Number.isSafeInteger(value) || value < minimum) throw new
Error(`Invalid ${name}`);
  return value;
};
/** Strict boundary adapter for the accepted candidate pack; it never repairs content. */
```

```
export function adaptLevel(input: unknown): LevelDefinition {
  const raw = object(input, 'level');
  if (raw.gameplayMode !== 'click-fill' || raw.inputMode !== 'tap') throw new Error('Only click-fill
levels are supported');
  const grid = object(raw.grid, 'grid');
  const challenge = object(raw.challenge, 'challenge');
  const revive = object(challenge.revive, 'revive');
  const solution = object(raw.solution, 'solution');
  if (array(solution.acceptedAlternativeLayouts, 'alternative layouts').length !== 0) {
    throw new Error('Alternative answer layouts require an explicit adapter upgrade');
  }
  if (!['none', 'moves', 'time'].includes(challenge.mode as string)) throw new Error('Invalid
challenge mode');
  const targetByCell = object(solution.targetByCell, 'targetByCell');
  const cells = array(raw.cells, 'cells').map(value => {
    const cell = object(value, 'cell');
    const id = string(cell.id, 'cell id');
    const targetHex = hex(cell.targetHex);
    if (hex(targetByCell[id]) !== targetHex) throw new Error('Cell and solution disagree');
    return { id, x: integer(cell.x, 'cell x'), y: integer(cell.y, 'cell y'), targetHex };
  });
  const tiles = array(raw.tiles, 'tiles').map(value => {
    const tile = object(value, 'tile');
    return { id: string(tile.id, 'tile id'), hex: hex(tile.hex) };
  });
  const paths = array(raw.paths, 'paths').map(value => {
    const path = object(value, 'path');
    return { id: string(path.id, 'path id'), cellIds: array(path.cellIds, 'path cells').map(v =>
string(v, 'path cell')) };
  });
  const initialBoardRaw = object(raw.initialBoard, 'initialBoard');
  const initialBoard: Board = Object.create(null) as Board;
  for (const cell of cells) {
    const value = initialBoardRaw[cell.id];
    initialBoard[cell.id] = value === null ? null : string(value, 'board tile');
  }
  const level: LevelDefinition = {
    gameplayMode: 'click-fill', inputMode: 'tap',
    id: string(raw.id, 'level id'), number: integer(raw.number, 'level number', 1),
    title: string(raw.title, 'level title'), chapter: integer(raw.chapter, 'chapter', 1),
    chapterName: string(raw.chapterName, 'chapterName'), contentVersion: string(raw.contentVersion,
'contentVersion'),
    scoreEpoch: raw.scoreEpoch === undefined ? DEFAULT_SCORE_EPOCH : string(raw.scoreEpoch, 'score
epoch'),
    grid: { columns: integer(grid.columns, 'columns', 1), rows: integer(grid.rows, 'rows', 1) },
    cells, paths, anchors: array(raw.anchors, 'anchors').map(v => string(v, 'anchor')),
    tiles, initialBoard, initialPool: array(raw.initialPool, 'pool').map(v => string(v, 'pool
tile')),
    trayHomeByTile: Object.fromEntries(Object.entries(object(raw.trayHomeByTile,
'trayHomeByTile')).map(([id, index]) => [id, integer(index, 'home slot')])),
    tutorialText: typeof raw.tutorialText === 'string' ? raw.tutorialText : '',
    challenge: {
      mode: 'time',
      timeLimitSeconds: challenge.mode === 'time' ? integer(challenge.timeLimitSeconds, 'time
limit', 1)
        : defaultTimeLimitSeconds(integer(raw.number, 'number', 1), integer(raw.chapter, 'chapter',
1), array(raw.initialPool, 'pool').length),
      revive: { maxPerAttempt: 2, addSeconds: 30 },
    },
```

```
  };
  if (!cells.length || new Set(cells.map(c => c.id)).size !== cells.length) throw new
Error('Duplicate or empty cells');
  if (new Set(cells.map(c => `${c.x}:${c.y}`)).size !== cells.length) throw new Error('Duplicate
coordinates');
  if (cells.some(c => c.x >= level.grid.columns || c.y >= level.grid.rows)) throw new Error('Cell
outside grid');
  if (new Set(tiles.map(t => t.id)).size !== tiles.length) throw new Error('Duplicate tile IDs');
  if (new Set(paths.map(p => p.id)).size !== paths.length) throw new Error('Duplicate path IDs');
  if (new Set(level.anchors).size !== level.anchors.length) throw new Error('Duplicate anchors');
  if (paths.some(p => !p.cellIds.length || p.cellIds.some(id => !own(initialBoard, id)))) throw new
Error('Invalid path reference');
  if (Object.keys(initialBoardRaw).length !== cells.length || Object.keys(targetByCell).length !==
cells.length) throw new Error('Unexpected board cell');
  if (level.anchors.some(id => !own(initialBoard, id))) throw new Error('Invalid anchor reference');
  const allTiles = [...Object.values(initialBoard), ...level.initialPool].filter((v): v is string =>
v !== null);
  const tileIds = new Set(tiles.map(t => t.id));
  if (allTiles.length !== tiles.length || new Set(allTiles).size !== tiles.length ||
allTiles.some(id => !tileIds.has(id))) {
    throw new Error('Initial tile conservation failed');
  }
  if (cells.length !== tiles.length) throw new Error('One tile per target cell required');
  const targets = cells.map(c => c.targetHex).sort();
  const colours = tiles.map(t => t.hex).sort();
  if (targets.some((colour, i) => colour !== colours[i])) throw new Error('Target colour multiset
mismatch');
  if (level.anchors.some(id => colourAt(level, initialBoard, id) !== cells.find(c => c.id ===
id)!.targetHex)) {
    throw new Error('Initial anchor is not correct');
  }
  if (challenge.mode === 'time' && (revive.maxPerAttempt !== 2 || revive.addSeconds !== 30)) throw
new Error('Invalid timed revival policy');
  if (cells.some(cell => !level.anchors.includes(cell.id) && initialBoard[cell.id] !== null)) throw
new Error('Only anchor tiles may be prefilled');
  const movableCount = cells.length - level.anchors.length;
  if (movableCount < 1 || level.initialPool.length !== movableCount) throw new Error('All movable
tiles must start in the tray');
  if (Object.keys(level.trayHomeByTile).length !== movableCount || level.initialPool.some((id,
index) => level.trayHomeByTile[id] !== index)) throw new Error('Invalid permanent tray homes');
  if (challenge.minimumMovesMetric !== 'tray-to-empty-board-placements' || challenge.minimumMoves !
== movableCount) throw new Error('Invalid minimum placement metric');
  if (level.challenge.timeLimitSeconds < 45 || level.challenge.timeLimitSeconds > 240) throw new
Error('Time limit outside supported range');
  return level;
}
function colourAt(level: LevelDefinition, board: Board, cellId: string): string | null {
  const tileId = board[cellId];
  return tileId == null ? null : level.tiles.find(tile => tile.id === tileId)?.hex ?? null;
}
export function isSolved(level: LevelDefinition, board: Board, pool: (string | null)[]): boolean {
  return pool.length === level.initialPool.length && pool.every(id => id === null)
    && level.cells.every(cell => colourAt(level, board, cell.id) === cell.targetHex);
}
export function isLocked(state: GameState, cellId: string): boolean {
  return state.level.anchors.includes(cellId) || state.attempt.auxiliaryLocks.includes(cellId);
}
function makeAttempt(level: LevelDefinition, attemptId: string): Attempt {
  string(attemptId, 'attempt ID');
  return {
    rulesVersion: RULES_VERSION, id: attemptId, levelId: level.id, levelContentVersion:
level.contentVersion, revision: 0,
    status: isSolved(level, level.initialBoard, level.initialPool) ? 'Won' : 'Playing',
    board: { ...level.initialBoard }, pool: [...level.initialPool], auxiliaryLocks: [],
    initialTimeMs: level.challenge.timeLimitSeconds * 1000, remainingTimeMs:
level.challenge.timeLimitSeconds * 1000, elapsedMs: 0,
    movesSpent: 0, totalTimeGrantedMs: 0, score: null, pointsEarned: 0, undoStack: [], propUses:
emptyInventory(),
```

```
    revivesUsed: 0, propRestockAdsGranted: 0, reviveAdsGranted: 0,
    pendingRewardRequestId: null, previousStatusBeforeAd: null, preview: null,
  };
}
function validateInventory(inventory: Inventory): Inventory {
  return { locate: integer(inventory.locate, 'locate stock'), peek: integer(inventory.peek, 'peek
stock'), addTime: integer(inventory.addTime, 'addTime stock') };
}
export function createGame(level: LevelDefinition, attemptId: string, inventory: Inventory =
{ ...INITIAL_INVENTORY }): GameState {
  return { level, attempt: makeAttempt(level, attemptId), wallet: { points: 0, levelScores: {},
inventory: validateInventory(inventory), rewardLedger: {}, pendingRevives: [] } };
}
function restoreLayout(level: LevelDefinition, input: unknown): Pick<UndoEntry, 'board' | 'pool' |
'auxiliaryLocks'> {
  const raw = object(input, 'layout');
  const rawBoard = object(raw.board, 'saved board');
  if (Object.keys(rawBoard).length !== level.cells.length) throw new Error('Saved board size
mismatch');
  const board: Board = {};
  for (const cell of level.cells) board[cell.id] = rawBoard[cell.id] === null ? null :
string(rawBoard[cell.id], 'saved tile');
  const pool = array(raw.pool, 'saved tray').map(value => value === null ? null : string(value,
'saved tray tile'));
  const auxiliaryLocks = array(raw.auxiliaryLocks, 'saved locks').map(value => string(value, 'saved
lock'));
  if (pool.length !== level.initialPool.length) throw new Error('Saved tray size mismatch');
  const ids = [...Object.values(board), ...pool].filter((id): id is string => id !== null);
  const tileIds = new Set(level.tiles.map(tile => tile.id));
  if (ids.length !== tileIds.size || new Set(ids).size !== tileIds.size || ids.some(id => !
tileIds.has(id))) throw new Error('Saved tile conservation failed');
  if (level.anchors.some(id => board[id] !== level.initialBoard[id])) throw new Error('Saved anchor
moved');
  if (new Set(auxiliaryLocks).size !== auxiliaryLocks.length || auxiliaryLocks.some(id => !
own(board, id) || level.anchors.includes(id) || colourAt(level, board, id) !== level.cells.find(cell
=> cell.id === id)!.targetHex)) throw new Error('Saved lock invalid');
  return { board, pool, auxiliaryLocks };
}
/** Restores only this rules/content version. Old attempts must be discarded, never reinterpreted.
*/
export function restoreAttempt(level: LevelDefinition, input: unknown): Attempt {
  const raw = object(input, 'attempt');
  if (raw.rulesVersion !== RULES_VERSION || raw.levelId !== level.id || raw.levelContentVersion !==
level.contentVersion) throw new Error('Incompatible attempt rules or level version');
  const layout = restoreLayout(level, raw);
  const propUses = validateInventory(object(raw.propUses, 'saved prop uses') as Inventory);
  for (const prop of Object.keys(PROP_LIMITS) as Prop[]) if (propUses[prop] > PROP_LIMITS[prop])
throw new Error('Saved prop limit exceeded');
  const initialTimeMs = integer(raw.initialTimeMs, 'saved initial time', 1);
  const remainingTimeMs = integer(raw.remainingTimeMs, 'saved remaining time');
  const elapsedMs = integer(raw.elapsedMs, 'saved elapsed time');
  const totalTimeGrantedMs = integer(raw.totalTimeGrantedMs, 'saved granted time');
  const movesSpent = integer(raw.movesSpent, 'saved spent moves');
  if (initialTimeMs !== level.challenge.timeLimitSeconds * 1000) throw new Error('Saved initial time
mismatch');
  if (remainingTimeMs !== initialTimeMs + totalTimeGrantedMs - elapsedMs) throw new Error('Saved
time accounting mismatch');
  const score = raw.score === null ? null : integer(raw.score, 'saved score', 100);
  if (score !== null && score > 200) throw new Error('Saved score too high');
  const pointsEarned = integer(raw.pointsEarned, 'saved points earned');
  if (pointsEarned > 200 || (score === null && pointsEarned !== 0)) throw new Error('Saved score
award invalid');
  const revivesUsed = integer(raw.revivesUsed, 'saved revival count');
  const propRestockAdsGranted = integer(raw.propRestockAdsGranted, 'saved restock count');
  const reviveAdsGranted = integer(raw.reviveAdsGranted, 'saved rewarded revival count');
  if (totalTimeGrantedMs !== (propUses.addTime + revivesUsed) * 30000) throw new Error('Saved
granted time mismatch');
  if (revivesUsed > level.challenge.revive.maxPerAttempt || propRestockAdsGranted > 1 ||
reviveAdsGranted > revivesUsed) throw new Error('Saved reward limit exceeded');
  const status = raw.status as AttemptStatus;
```

```
  if (!['Playing', 'Won', 'Failed', 'AdPending', 'PreviewingReference'].includes(status)) throw new
Error('Saved status invalid');
  const solved = isSolved(level, layout.board, layout.pool);
  const previousStatusBeforeAd = raw.previousStatusBeforeAd === null ? null :
raw.previousStatusBeforeAd as 'Playing' | 'Failed' | 'Won';
  const pendingRewardRequestId = raw.pendingRewardRequestId === null ? null :
string(raw.pendingRewardRequestId, 'pending request');
  const baseStatus = status === 'AdPending' ? previousStatusBeforeAd : status ===
'PreviewingReference' ? 'Playing' : status;
  if (!['Playing', 'Won', 'Failed'].includes(baseStatus ?? '')) throw new Error('Saved base status
invalid');
  if (solved !== (baseStatus === 'Won') || (baseStatus === 'Failed' && remainingTimeMs !== 0) ||
(baseStatus === 'Playing' && remainingTimeMs === 0)) throw new Error('Saved status disagrees with
board or budget');
  if ((baseStatus === 'Won') !== (score !== null)) throw new Error('Saved score status mismatch');
  if ((status === 'AdPending') !== (pendingRewardRequestId !== null && previousStatusBeforeAd !==
null)) throw new Error('Saved ad state invalid');
  let preview: Attempt['preview'] = null;
  if (status === 'PreviewingReference') {
    const item = object(raw.preview, 'saved preview');
    const pathId = string(item.pathId, 'saved preview path');
    if (!level.paths.some(path => path.id === pathId) || typeof item.expiresAtMs !== 'number' || !
Number.isFinite(item.expiresAtMs)) throw new Error('Saved preview invalid');
    preview = { pathId, expiresAtMs: item.expiresAtMs };
  } else if (raw.preview !== null) throw new Error('Unexpected saved preview');
  const undoStack = array(raw.undoStack, 'saved history').map(value => {
    const entry = object(value, 'saved history entry');
    if (!['place', 'move', 'returnToTray', 'locate'].includes(entry.kind as string)) throw new
Error('Incompatible history action');
    return { kind: entry.kind as UndoEntry['kind'], ...restoreLayout(level, entry) };
  });
  return { ...layout, rulesVersion: RULES_VERSION, id: string(raw.id, 'saved attempt ID'), levelId:
level.id,
    levelContentVersion: level.contentVersion, revision: integer(raw.revision, 'saved revision'),
status,
    initialTimeMs, remainingTimeMs, elapsedMs, movesSpent, totalTimeGrantedMs, score, pointsEarned,
undoStack,
    propUses, revivesUsed, propRestockAdsGranted, reviveAdsGranted, pendingRewardRequestId,
previousStatusBeforeAd, preview };
}
/** The application may preserve wallet/progress separately when an incompatible attempt is
discarded. */
export function restoreGame(level: LevelDefinition, input: unknown): GameState {
  const raw = object(input, 'saved game');
  const attempt = restoreAttempt(level, raw.attempt);
  const wallet = restoreWallet(raw.wallet);
  if (attempt.pendingRewardRequestId) {
    const request = wallet.rewardLedger[attempt.pendingRewardRequestId];
    if (!request || request.status !== 'pending' || request.attemptId !== attempt.id ||
request.levelId !== level.id || request.levelContentVersion !== level.contentVersion) throw new
Error('Saved pending reward mismatch');
  }
  return { level, attempt, wallet };
}
/** Validates earned rights independently of the attempt. Legacy move entitlements become seconds.
*/
export function restoreWallet(input: unknown, legacy = false): Wallet {
  const rawWallet = object(input, 'saved wallet');
  const rawInventory = object(rawWallet.inventory, 'saved inventory');
  const inventory = validateInventory({ ...rawInventory,
    addTime: rawInventory.addTime ?? (legacy ? rawInventory.addMoves : undefined),
  } as Inventory);
  const points = legacy && rawWallet.points === undefined ? 0 : integer(rawWallet.points, 'saved
points');
  const levelScores: Record<string, LevelScore> = {};
  for (const [id, value] of Object.entries(legacy && rawWallet.levelScores === undefined ? {} :
object(rawWallet.levelScores, 'level scores'))) {
    const item = object(value, 'level score');
    const bestScore = integer(item.bestScore, 'best score', 100);
    if (!/^L\d{3}$/.test(id) || bestScore > 200 || typeof item.doubleScoreClaimed !== 'boolean')
throw new Error('Saved level score invalid');
```

```
    const scoreEpoch = item.scoreEpoch === undefined ? {} : { scoreEpoch: string(item.scoreEpoch,
'saved score epoch') };
    levelScores[id] = { ...scoreEpoch, bestScore, doubleScoreClaimed: item.doubleScoreClaimed };
  }
  const rewardLedger: Record<string, RewardRequest> = {};
  for (const [id, value] of Object.entries(object(rawWallet.rewardLedger, 'reward ledger'))) {
    const item = object(value, 'reward request');
    if (!validId(id) || item.id !== id || !['pending', 'granted', 'cancelled',
'failed'].includes(item.status as string) || !['revive', 'restock',
'doubleScore'].includes(item.rewardType as string)) throw new Error('Saved reward request invalid');
    const rewardType = item.rewardType as RewardRequest['rewardType'];
    const prop = rewardType === 'restock' ? (legacy && item.prop === 'addMoves' ? 'addTime' :
item.prop) as Prop : null;
    const oldAmount = integer(item.amount, 'saved reward amount', 1);
    const amount = legacy && rewardType === 'revive' ? 30 : oldAmount;
    if (rewardType === 'restock' && (!own(PROP_LIMITS, prop!) || amount !== 1)) throw new
Error('Saved restock reward invalid');
    if (rewardType === 'revive' && amount !== 30) throw new Error('Saved revival amount invalid');
    if (rewardType === 'doubleScore' && amount > 200) throw new Error('Saved bonus amount invalid');
    rewardLedger[id] = { id, attemptId: string(item.attemptId, 'reward attempt'), levelId:
string(item.levelId, 'reward level'),
      levelContentVersion: string(item.levelContentVersion, 'reward content version'), rewardType,
prop, amount,
      ...(item.scoreEpoch === undefined ? {} : { scoreEpoch: string(item.scoreEpoch, 'reward score
epoch') }), status: item.status as RewardRequest['status'] };
  }
  if (Object.values(rewardLedger).filter(item => item.status === 'pending').length > 1) throw new
Error('Multiple pending rewards');
  const pendingRevives = array(rawWallet.pendingRevives, 'pending revivals').map(value => {
    const item = object(value, 'pending revival');
    const requestId = string(item.requestId, 'pending revival request');
    const request = rewardLedger[requestId];
    const originAttemptId = string(item.originAttemptId, 'pending revival attempt');
    const oldAmount = integer(item.amount, 'pending revival amount', 1);
    const amount = legacy ? 30 : oldAmount;
    if (!request || request.status !== 'granted' || request.rewardType !== 'revive' ||
request.attemptId !== originAttemptId || request.amount !== amount) throw new Error('Saved pending
revival mismatch');
    return { requestId, originAttemptId, amount };
  });
  if (new Set(pendingRevives.map(item => item.requestId)).size !== pendingRevives.length) throw new
Error('Duplicate pending revival');
  return { points, levelScores, inventory, rewardLedger, pendingRevives };
}
/** Application owns sequential level unlocks and one-time starter/chapter grants. */
export function startLevel(previous: GameState, level: LevelDefinition, attemptId: string):
GameState {
  if (attemptId === previous.attempt.id || Object.values(previous.wallet.rewardLedger).some(request
=> request.attemptId === attemptId)) {
    throw new Error('A new attempt ID is required');
  }
  return { level, attempt: makeAttempt(level, attemptId), wallet: cloneWallet(previous.wallet) };
}
/** Resume presentation-independent state. Unverified ad requests remain in the ledger. */
export function recoverInterruptedGame(state: GameState): GameState {
  if (state.attempt.status !== 'PreviewingReference' && state.attempt.status !== 'AdPending') return
state;
  const next = clone(state);
  if (next.attempt.status === 'PreviewingReference') {
    next.attempt.preview = null;
    next.attempt.status = 'Playing';
  } else {
    next.attempt.status = next.attempt.previousStatusBeforeAd ?? 'Failed';
    next.attempt.previousStatusBeforeAd = null;
    next.attempt.pendingRewardRequestId = null;
```

```
  }
  next.attempt.revision++;
  return next;
}
function cloneWallet(wallet: Wallet): Wallet {
  const rewardLedger: Record<string, RewardRequest> = {};
  for (const id of Object.keys(wallet.rewardLedger)) rewardLedger[id] = { ...wallet.rewardLedger[id]
};
  return { points: wallet.points, levelScores:
Object.fromEntries(Object.entries(wallet.levelScores).map(([id, score]) => [id, { ...score }])),
inventory: { ...wallet.inventory }, rewardLedger, pendingRevives: wallet.pendingRevives.map(item =>
({ ...item })) };
}
function clone(state: GameState): GameState {
  const attempt = state.attempt;
  return {
    level: state.level,
    attempt: { ...attempt, board: { ...attempt.board }, pool: [...attempt.pool], auxiliaryLocks:
[...attempt.auxiliaryLocks],
      undoStack: [...attempt.undoStack], propUses: { ...attempt.propUses }, preview: attempt.preview
? { ...attempt.preview } : null },
    wallet: cloneWallet(state.wallet),
  };
}
function snapshot(state: GameState, kind: UndoEntry['kind']): void {
  state.attempt.undoStack.push({ kind, board: { ...state.attempt.board }, pool:
[...state.attempt.pool], auxiliaryLocks: [...state.attempt.auxiliaryLocks] });
}
function rejected(state: GameState, reason: string): Transition {
  return { state, changed: false, reason, events: [] };
}
function committed(state: GameState, events: DomainEvent[] = []): Transition {
  state.attempt.revision++;
  return { state, changed: true, events };
}
export function defaultTimeLimitSeconds(number: number, chapter: number, movableCount: number):
number {
  return number === 1 ? 45 : Math.min(240, Math.max(45, 45 + movableCount * (chapter <= 2 ? 6 :
chapter <= 4 ? 5 : 4)));
}
/** 100 base + 0..50 placement efficiency + 0..50 remaining original time, with aid penalties. */
export function calculateScore(state: GameState): number {
  const attempt = state.attempt;
  const minimum = state.level.initialPool.length;
  const efficiency = Math.round(50 * Math.min(1, minimum / Math.max(minimum, attempt.movesSpent)));
  const time = Math.round(50 * Math.min(1, Math.max(0, attempt.remainingTimeMs /
attempt.initialTimeMs)));
  const aids = Object.values(attempt.propUses).reduce((sum, uses) => sum + uses, 0) +
attempt.revivesUsed;
  return Math.max(100, 100 + efficiency + time - aids * 10);
}
function effectiveScoreEpoch(value: { scoreEpoch?: string }): string {
  return value.scoreEpoch ?? DEFAULT_SCORE_EPOCH;
}
export function canClaimDoubleScore(state: GameState): boolean {
  const record = state.wallet.levelScores[state.level.id];
  return state.attempt.status === 'Won' && state.attempt.pointsEarned > 0 && !!record
    && effectiveScoreEpoch(record) === effectiveScoreEpoch(state.level) && !
record.doubleScoreClaimed;
}
function resolveBoard(state: GameState, spendMove: boolean): DomainEvent[] {
  const attempt = state.attempt;
```

```
  if (spendMove) attempt.movesSpent++;
  if (isSolved(state.level, attempt.board, attempt.pool)) {
    attempt.status = 'Won';
    attempt.score = calculateScore(state);
    const saved = state.wallet.levelScores[state.level.id];
    const previous = saved && effectiveScoreEpoch(saved) === effectiveScoreEpoch(state.level) ?
saved : undefined;
    const bestScore = Math.max(previous?.bestScore ?? 0, attempt.score);
    attempt.pointsEarned = bestScore - (previous?.bestScore ?? 0);
    state.wallet.points += attempt.pointsEarned;
    state.wallet.levelScores[state.level.id] = { scoreEpoch: effectiveScoreEpoch(state.level),
bestScore, doubleScoreClaimed: previous?.doubleScoreClaimed ?? false };
    return [{ type: 'won', levelId: state.level.id, attemptId: attempt.id, score: attempt.score,
pointsEarned: attempt.pointsEarned }];
  }
  attempt.status = 'Playing';
  return [];
}
function propReason(state: GameState, prop: Prop): string | undefined {
  if (state.attempt.status !== 'Playing') return '当前状态不能使用道具';
  if (state.attempt.propUses[prop] >= PROP_LIMITS[prop]) return '本次尝试已达道具使用上限';
  if (state.wallet.inventory[prop] <= 0) return '道具库存不足';
  return undefined;
}
function spendProp(state: GameState, prop: Prop): void {
  state.wallet.inventory[prop]--;
  state.attempt.propUses[prop]++;
}
function validId(value: string): boolean {
  return typeof value === 'string' && value.length > 0 && value !== '__proto__' && value !==
'constructor' && value !== 'prototype';
}
/** Returns a new atomic snapshot. Persist the entire snapshot together, before animations. */
export function reduceGame(state: GameState, action: Action): Transition {
  const attempt = state.attempt;
  if (action.type === 'restart') {
    if (!validId(action.attemptId) || action.attemptId === attempt.id ||
Object.values(state.wallet.rewardLedger).some(r => r.attemptId === action.attemptId)) {
      return rejected(state, '重开必须使用新的尝试 ID');
    }
    const next = startLevel(state, state.level, action.attemptId);
    return committed(next);
  }
  if (action.type === 'settleReward') return settleReward(state, action.requestId, action.outcome);
  if (action.type === 'beginReward') return beginReward(state, action);
  if (action.type === 'applyPendingRevive') {
    if (attempt.status !== 'Failed' || attempt.revivesUsed >=
state.level.challenge.revive.maxPerAttempt) return rejected(state, '当前不能使用待用复活权益');
    const item = state.wallet.pendingRevives.find(r => r.requestId === action.requestId);
    if (!item) return rejected(state, '待用复活权益不存在');
    const next = clone(state);
    next.wallet.pendingRevives = next.wallet.pendingRevives.filter(r => r.requestId !==
action.requestId);
    applyRevive(next, item.amount);
    return committed(next);
  }
  if (action.type === 'endPeek' || action.type === 'expirePeek' || action.type === 'suspend') {
```

```
    if (attempt.status !== 'PreviewingReference') return rejected(state, '没有正在显示的参考');
    if (action.type === 'expirePeek' && (!Number.isFinite(action.nowMs) || action.nowMs <
attempt.preview!.expiresAtMs)) return rejected(state, '参考尚未到期');
    const next = clone(state);
    next.attempt.status = 'Playing';
    next.attempt.preview = null;
    return committed(next);
  }
  if (action.type === 'purchaseProp') {
    if (attempt.status === 'AdPending' || attempt.status === 'PreviewingReference') return
rejected(state, '请先结束当前操作');
    if (!own(PROP_PRICES, action.prop)) return rejected(state, '无效道具');
    const price = PROP_PRICES[action.prop];
    if (state.wallet.points < price) return rejected(state, '积分不足');
    const next = clone(state);
    next.wallet.points -= price;
    next.wallet.inventory[action.prop]++;
    return committed(next);
  }
  if (attempt.status !== 'Playing') return rejected(state, '当前状态不能操作棋盘');
  if (action.type === 'elapse') {
    if (!Number.isSafeInteger(action.deltaMs) || action.deltaMs <= 0) return rejected(state, '无效计
时增量');
    const next = clone(state);
    const elapsed = Math.min(action.deltaMs, next.attempt.remainingTimeMs);
    next.attempt.remainingTimeMs -= elapsed;
    next.attempt.elapsedMs += elapsed;
    if (next.attempt.remainingTimeMs === 0) {
      next.attempt.status = 'Failed';
      return committed(next, [{ type: 'failed', levelId: next.level.id, attemptId:
next.attempt.id }]);
    }
    return committed(next);
  }
  if (action.type === 'move') {
    const from = action.fromCellId, to = action.toCellId;
    if (from === to || !own(attempt.board, from) || !own(attempt.board, to) || isLocked(state, from)
|| isLocked(state, to) || !attempt.board[from] || attempt.board[to] !== null) {
      return rejected(state, '请选择色块和另一个空格');
    }
    const next = clone(state);
    snapshot(next, 'move');
    next.attempt.board[to] = next.attempt.board[from];
    next.attempt.board[from] = null;
    return committed(next, resolveBoard(next, true));
  }
  if (action.type === 'returnToTray') {
    const from = action.fromCellId;
    if (!own(attempt.board, from) || isLocked(state, from) || !attempt.board[from]) return
rejected(state, '请选择可取回的色块');
    const home = state.level.trayHomeByTile[attempt.board[from]!];
    const index = action.toPoolIndex !== undefined ? action.toPoolIndex
      : attempt.pool[home] === null ? home : attempt.pool.findIndex(tile => tile === null);
    if (!Number.isInteger(index) || index < 0 || index >= attempt.pool.length || attempt.pool[index]
!== null) return rejected(state, '请选择待选区的空槽');
    const next = clone(state);
    snapshot(next, 'returnToTray');
```

```
    next.attempt.pool[index] = next.attempt.board[from];
    next.attempt.board[from] = null;
    return committed(next, resolveBoard(next, false));
  }
  if (action.type === 'place') {
    const index = action.poolIndex, target = action.toCellId;
    if (!Number.isInteger(index) || index < 0 || index >= attempt.pool.length || !
attempt.pool[index] || !own(attempt.board, target) || attempt.board[target] !== null ||
isLocked(state, target)) {
      return rejected(state, '请把待选色块放入空格');
    }
    const next = clone(state);
    snapshot(next, 'place');
    next.attempt.board[target] = next.attempt.pool[index];
    next.attempt.pool[index] = null;
    return committed(next, resolveBoard(next, true));
  }
  if (action.type === 'undo') {
    const entry = attempt.undoStack[attempt.undoStack.length - 1];
    if (!entry) return rejected(state, '还没有可以撤销的操作');
    const next = clone(state);
    next.attempt.undoStack.pop();
    next.attempt.board = { ...entry.board };
    next.attempt.pool = [...entry.pool];
    next.attempt.auxiliaryLocks = [...entry.auxiliaryLocks];
    return committed(next);
  }
  if (action.type === 'addTime') {
    const reason = propReason(state, 'addTime');
    if (reason) return rejected(state, reason);
    const next = clone(state);
    spendProp(next, 'addTime');
    next.attempt.remainingTimeMs += 30000;
    next.attempt.totalTimeGrantedMs += 30000;
    return committed(next);
  }
  if (action.type === 'peek') {
    const reason = propReason(state, 'peek');
    if (reason) return rejected(state, reason);
    if (!state.level.paths.some(p => p.id === action.pathId) || !Number.isFinite(action.nowMs))
return rejected(state, '请选择一条有效色路');
    const next = clone(state);
    spendProp(next, 'peek');
    next.attempt.status = 'PreviewingReference';
    next.attempt.preview = { pathId: action.pathId, expiresAtMs: action.nowMs + 3000 };
    return committed(next);
  }
  if (action.type === 'locate') {
    const reason = propReason(state, 'locate');
    if (reason) return rejected(state, reason);
    const candidates = state.level.cells.filter(c => !isLocked(state, c.id) && colourAt(state.level,
attempt.board, c.id) !== c.targetHex);
    const target = candidates[0];
    if (!target) return rejected(state, '没有可归位的色块');
```

```
    const sourceCell = state.level.cells.find(c => !isLocked(state, c.id) && colourAt(state.level,
attempt.board, c.id) === target.targetHex);
    const sourcePool = attempt.pool.findIndex(id => id !== null && state.level.tiles.find(t => t.id
=== id)?.hex === target.targetHex);
    if (!sourceCell && sourcePool < 0) return rejected(state, '没有可用的目标色块');
    const next = clone(state);
    snapshot(next, 'locate');
    const displacedTile = next.attempt.board[target.id];
    // Extract the correct tile first. Its old tray position becomes available before any
    // displaced tile is returned, so arbitrary tray rearrangements never require a swap.
    const correctTile = sourceCell ? next.attempt.board[sourceCell.id] :
next.attempt.pool[sourcePool];
    if (sourceCell) next.attempt.board[sourceCell.id] = null;
    else next.attempt.pool[sourcePool] = null;
    if (displacedTile !== null) {
      const home = state.level.trayHomeByTile[displacedTile];
      const slot = next.attempt.pool[home] === null ? home : next.attempt.pool.findIndex(tile =>
tile === null);
      if (slot < 0) return rejected(state, '没有可用的待选空槽');
      next.attempt.pool[slot] = displacedTile;
    }
    next.attempt.board[target.id] = correctTile;
    next.attempt.auxiliaryLocks.push(target.id);
    spendProp(next, 'locate');
    return committed(next, resolveBoard(next, false));
  }
  return rejected(state, '未知动作');
}
function beginReward(state: GameState, action: Extract<Action, { type: 'beginReward' }>): Transition
{
  const attempt = state.attempt;
  if (!validId(action.requestId) || own(state.wallet.rewardLedger, action.requestId)) return
rejected(state, '奖励请求 ID 已使用或无效');
  // A detached, unconfirmed request remains busy until the platform resolves it.
  if (Object.values(state.wallet.rewardLedger).some(r => r.status === 'pending')) return
rejected(state, '已有视频请求等待确认');
  if (action.rewardType === 'revive') {
    if (attempt.status !== 'Failed' || attempt.revivesUsed >=
state.level.challenge.revive.maxPerAttempt) return rejected(state, '当前不能申请视频复活');
  } else if (action.rewardType === 'doubleScore') {
    if (!canClaimDoubleScore(state)) return rejected(state, '当前不能申请双倍积分');
  } else if (action.rewardType === 'restock') {
    if (attempt.status !== 'Playing' || !action.prop || !own(PROP_LIMITS, action.prop)) return
rejected(state, '当前不能申请道具补货');
    if (attempt.propRestockAdsGranted >= 1) return rejected(state, '本次尝试已领取过道具补货');
    if (state.wallet.inventory[action.prop] !== 0 || attempt.propUses[action.prop] >=
PROP_LIMITS[action.prop]) return rejected(state, '当前道具无需或无法补货');
  } else return rejected(state, '无效奖励类型');
  const next = clone(state);
  next.wallet.rewardLedger[action.requestId] = {
    id: action.requestId, attemptId: attempt.id, levelId: state.level.id, levelContentVersion:
state.level.contentVersion,
    rewardType: action.rewardType,
    ...(action.rewardType === 'doubleScore' ? { scoreEpoch: effectiveScoreEpoch(state.level) } :
{}), amount: action.rewardType === 'revive' ? state.level.challenge.revive.addSeconds :
action.rewardType === 'doubleScore' ? attempt.pointsEarned : 1,
    prop: action.rewardType === 'restock' ? action.prop! : null, status: 'pending',
  };
  next.attempt.previousStatusBeforeAd = attempt.status as 'Playing' | 'Failed' | 'Won';
  next.attempt.status = 'AdPending';
  next.attempt.pendingRewardRequestId = action.requestId;
  return committed(next, [{ type: 'rewardRequested', requestId: action.requestId }]);
}
```

```
function applyRevive(state: GameState, amount: number): void {
  state.attempt.remainingTimeMs += amount * 1000;
  state.attempt.totalTimeGrantedMs += amount * 1000;
  state.attempt.revivesUsed++;
  state.attempt.status = 'Playing';
}
function settleReward(state: GameState, requestId: string, outcome: 'completed' | 'cancelled' |
'failed'): Transition {
  if (!['completed', 'cancelled', 'failed'].includes(outcome)) return rejected(state, '无效视频结算结
果');
  const request = own(state.wallet.rewardLedger, requestId) ? state.wallet.rewardLedger[requestId] :
undefined;
  if (!request || request.status !== 'pending') return rejected(state, '奖励请求不存在或已经处理');
  const next = clone(state);
  const sameAttempt = request.attemptId === next.attempt.id && request.levelId === next.level.id &&
request.levelContentVersion === next.level.contentVersion;
  if (sameAttempt && next.attempt.pendingRewardRequestId === requestId) {
    next.attempt.status = next.attempt.previousStatusBeforeAd ?? 'Failed';
    next.attempt.previousStatusBeforeAd = null;
    next.attempt.pendingRewardRequestId = null;
  }
  next.wallet.rewardLedger[requestId].status = outcome === 'completed' ? 'granted' : outcome;
  if (outcome !== 'completed') return committed(next);
  let deferred = false;
  if (request.rewardType === 'doubleScore') {
    const entry = next.wallet.levelScores[request.levelId];
    const matchingEpoch = !!entry && effectiveScoreEpoch(entry) === effectiveScoreEpoch(request);
    if (!matchingEpoch || !entry.doubleScoreClaimed) {
      // A late confirmed video from an old puzzle design keeps its earned bonus, but cannot
      // consume the new design's independent double-score qualification.
      next.wallet.points += request.amount;
      if (matchingEpoch) entry.doubleScoreClaimed = true;
    }
  } else if (request.rewardType === 'restock') {
    next.wallet.inventory[request.prop!] += request.amount;
    if (sameAttempt) next.attempt.propRestockAdsGranted++;
  } else if (sameAttempt && next.attempt.status === 'Failed' && next.attempt.revivesUsed <
next.level.challenge.revive.maxPerAttempt) {
    applyRevive(next, request.amount);
    next.attempt.reviveAdsGranted++;
  } else {
    deferred = true;
    next.wallet.pendingRevives.push({ requestId, originAttemptId: request.attemptId, amount:
request.amount });
  }
  return committed(next, [{ type: 'rewardGranted', requestId, deferred }]);
}
import type { GameState } from '../core';
export interface PlayerSettings {
  sound: boolean;
  haptics: boolean;
  reducedMotion: boolean;
}
export function defaultSettings(): PlayerSettings {
  return { sound: true, haptics: true, reducedMotion: false };
}
```

```
export interface ProgressSave {
  version: 3;
  gameplayMode: 'click-fill';
  inputMode: 'tap';
  contentVersion: string;
  completed: string[];
  starterGranted: boolean;
  chapterGifts: number[];
  settings: PlayerSettings;
  game: GameState | null;
}
/** Keep the existing storage address so both old atomic copies can be migrated to version 3. */
export const SAVE_KEY = 'shisezhijian.click-fill.progress.v2';
export function emptyProgress(contentVersion: string): ProgressSave {
  return { version: 3, gameplayMode: 'click-fill', inputMode: 'tap', contentVersion,
    completed: [], starterGranted: false, chapterGifts: [], settings: defaultSettings(), game:
null };
}
/** Progress has no engine dependency; the storage adapter decides where it is written. */
export function completeLevel(save: ProgressSave, allIds: string[]): void {
  if (!save.game || save.game.attempt.status !== 'Won') return;
  const id = save.game.level.id;
  if (!save.completed.includes(id)) save.completed.push(id);
  if (id === 'L004' && !save.starterGranted) {
    save.game.wallet.inventory.locate += 3;
    save.game.wallet.inventory.peek += 2;
    save.game.wallet.inventory.addTime += 1;
    save.starterGranted = true;
  }
  for (let chapter = 1; chapter <= 5; chapter++) {
    const ids = allIds.slice((chapter - 1) * 20, chapter * 20);
    if (ids.length === 20 && ids.every(levelId => save.completed.includes(levelId))
      && !save.chapterGifts.includes(chapter)) {
      save.game.wallet.inventory.locate++;
      save.game.wallet.inventory.peek++;
      save.game.wallet.inventory.addTime++;
      save.chapterGifts.push(chapter);
    }
  }
}
export function unlockedThrough(save: ProgressSave, allIds: string[]): number {
  const firstUnfinished = allIds.findIndex(id => !save.completed.includes(id));
  return firstUnfinished < 0 ? allIds.length - 1 : firstUnfinished;
}
import { createGame, GameState, LevelDefinition, restoreGame, restoreWallet } from '../core';
import { defaultSettings, PlayerSettings, ProgressSave, unlockedThrough } from './Progress';
function record(value: unknown, name: string): Record<string, unknown> {
  if (!value || typeof value !== 'object' || Array.isArray(value)) throw new Error(`Invalid ${name}
`);
  return value as Record<string, unknown>;
}
/** Settings damage cannot erase otherwise valid earned progress or inventory. */
```

```
function restoreSettings(value: unknown): PlayerSettings {
  const defaults = defaultSettings();
  if (!value || typeof value !== 'object' || Array.isArray(value)) return defaults;
  const raw = value as Record<string, unknown>;
  return {
    sound: typeof raw.sound === 'boolean' ? raw.sound : defaults.sound,
    haptics: typeof raw.haptics === 'boolean' ? raw.haptics : defaults.haptics,
    reducedMotion: typeof raw.reducedMotion === 'boolean' ? raw.reducedMotion :
defaults.reducedMotion,
  };
}
/**
 * Extracts global player rights from supported save envelopes and binds them to current content.
 * The old attempt is never interpreted using new rules. Invalid wallets throw so storage can try
 * another backup rather than silently replacing earned inventory or confirmed rewards with zero.
 */
export function restoreProgress(input: unknown, contentVersion: string,
    levels: LevelDefinition[], attemptId: string): ProgressSave {
  if (!levels.length || !contentVersion) throw new Error('Current content is required');
  const raw = record(input, 'progress');
  if (raw.version !== 1 && raw.version !== 2 && raw.version !== 3) throw new Error('Unsupported
progress version');
  if (typeof raw.contentVersion !== 'string' || !raw.contentVersion) throw new Error('Invalid saved
content version');
  if (typeof raw.starterGranted !== 'boolean' || !Array.isArray(raw.completed) || !
Array.isArray(raw.chapterGifts)) throw new Error('Invalid global progress');
  const allIds = levels.map(level => level.id);
  const ids = new Set(allIds);
  const completed = raw.completed as unknown[];
  const chapterGifts = raw.chapterGifts as unknown[];
  if (completed.some(id => typeof id !== 'string' || !ids.has(id)) || new Set(completed).size !==
completed.length) throw new Error('Invalid completed levels');
  if (chapterGifts.some(chapter => typeof chapter !== 'number' || !Number.isInteger(chapter) ||
chapter < 1 || chapter > 5)
      || new Set(chapterGifts).size !== chapterGifts.length) throw new Error('Invalid chapter gift
markers');
  const save: ProgressSave = {
    version: 3, gameplayMode: 'click-fill', inputMode: 'tap', contentVersion,
    completed: [...completed] as string[], starterGranted: raw.starterGranted,
    chapterGifts: [...chapterGifts] as number[], settings: restoreSettings(raw.settings), game:
null,
  };
  if (raw.game === null) {
    // A pristine save has no wallet yet. A played save without one cannot be reconstructed safely.
    if (save.completed.length || save.starterGranted || save.chapterGifts.length) throw new
Error('Played progress is missing its wallet');
    return save;
  }
  const rawGame = record(raw.game, 'saved game');
  const unlocked = unlockedThrough(save, allIds);
  const savedLevelId = rawGame.level && typeof rawGame.level === 'object'
    ? (rawGame.level as Record<string, unknown>).id : undefined;
  const savedIndex = levels.findIndex(level => level.id === savedLevelId);
  const level = levels[savedIndex >= 0 && savedIndex <= unlocked ? savedIndex : unlocked];
  const fresh = createGame(level, attemptId);
  // Validate the wallet independently of any damaged/obsolete board or attempt metadata.
  const wallet = restoreWallet(rawGame.wallet, raw.version !== 3);
  const walletOnly = restoreGame(level, { ...fresh, wallet });
  let restored: GameState | null = null;
```

```
  if (savedLevelId === level.id) {
    try { restored = restoreGame(level, { ...rawGame, wallet }); } catch { /* Only the attempt is
replaced below. */ }
  }
  if (restored) save.game = restored;
  else {
    const rawAttempt = rawGame.attempt;
    const oldAttemptId = rawAttempt && typeof rawAttempt === 'object'
      ? (rawAttempt as Record<string, unknown>).id : undefined;
    if (attemptId === oldAttemptId || Object.values(walletOnly.wallet.rewardLedger).some(request =>
request.attemptId === attemptId)) throw new Error('Replacement attempt ID must be fresh');
    save.game = walletOnly;
  }
  return save;
}
/** Shipping configuration. Empty ad unit IDs keep rewarded video explicitly unavailable. */
export const runtimeConfig = {
  version: '1.1.0',
  browserAccessibility: true,
  rewardedAdUnitId: '',
} as const;
import { sys } from 'cc';
import { emptyProgress, ProgressSave, SAVE_KEY } from '../application/Progress';
import { LevelDefinition, recoverInterruptedGame, reduceGame } from '../core';
import { restoreProgress } from '../application/RestoreProgress';
import { SnapshotStore, StorageBackend } from './SnapshotStore';
export class LocalSave {
  error = '';
  private snapshots: SnapshotStore<ProgressSave> | null = null;
  constructor(private readonly backend: StorageBackend = sys.localStorage) {}
  load(contentVersion: string, levels: LevelDefinition[]): ProgressSave {
    this.snapshots = new SnapshotStore(this.backend, SAVE_KEY, input =>
      restoreProgress(input, contentVersion, levels, `restored-${Date.now()}-${++this.recoveryId}
`));
    const latest = this.snapshots.load();
    this.error = this.snapshots.error;
    if (latest.value) return this.recoverColdStart(latest.value);
    // Earlier development snapshots contribute only validated global data, never old board rules.
    for (const key of ['shisezhijian.progress.v1', 'shisezhijian.progress.v1.backup']) {
      try {
        const value = this.backend.getItem(key);
        if (!value) continue;
        const parsed = restoreProgress(JSON.parse(value), contentVersion, levels,
          `restored-${Date.now()}-${++this.recoveryId}`);
        return this.recoverColdStart(parsed);
      } catch { this.error = '本地存档暂时不可用'; }
    }
    return emptyProgress(contentVersion);
  }
  private recoveryId = 0;
  private recoverColdStart(save: ProgressSave): ProgressSave {
    if (!save.game) return save;
    save.game = recoverInterruptedGame(save.game);
```

```
    // A cold-started process has no surviving SDK instance to resolve the old callback. Release
    // only unconfirmed requests so they cannot permanently block all future videos. Confirmed
    // grants and deferred revival entitlements remain exactly as saved; no reward is invented.
    for (const request of Object.values(save.game.wallet.rewardLedger)) {
      if (request.status === 'pending') {
        save.game = reduceGame(save.game, { type: 'settleReward', requestId: request.id, outcome:
'failed' }).state;
      }
    }
    return save;
  }
  write(save: ProgressSave): boolean {
    if (!this.snapshots) {
      this.error = '本地存档尚未读取，本次进度尚未保存';
      return false;
    }
    const saved = this.snapshots.write(save);
    this.error = this.snapshots.error;
    return saved;
  }
}
/** Synchronous storage shared by Cocos localStorage, tt storage and the test backend. */
export interface StorageBackend {
  getItem(key: string): string | null;
  setItem(key: string, value: string): void;
}
interface Snapshot<T> { key: string; revision: number; value: T; encoded: string; }
export interface SnapshotRead<T> {
  value: T | null;
  revision: number;
  warning: boolean;
  found: boolean;
}
/**
 * Two independently valid, monotonically versioned snapshots. An existing good copy is never
 * overwritten until a replacement has been written and read back byte-for-byte. The format is
 * an integrity/compatibility envelope, not an anti-cheat signature or a cloud save.
 */
export class SnapshotStore<T> {
  error = '';
  revision = 0;
  constructor(private readonly backend: StorageBackend, private readonly key: string,
    private readonly validate: (input: unknown) => T) {}
  private decode(key: string, encoded: string): Snapshot<T> {
    const raw = JSON.parse(encoded) as unknown;
    if (raw && typeof raw === 'object' && 'snapshotFormat' in raw) {
      const envelope = raw as Record<string, unknown>;
      if (envelope.snapshotFormat !== 1 || !Number.isSafeInteger(envelope.revision)
        || (envelope.revision as number) < 1 || !('payload' in envelope)) throw new Error('Invalid
snapshot envelope');
      return { key, revision: envelope.revision as number, value: this.validate(envelope.payload),
encoded };
    }
```

```
    // Earlier saves had a plain versioned progress payload, before the storage envelope existed.
    return { key, revision: 0, value: this.validate(raw), encoded };
  }
  private inspect(): { valid: Snapshot<T>[]; warning: boolean; found: boolean } {
    const valid: Snapshot<T>[] = [];
    let warning = false;
    let found = false;
    for (const key of [this.key, `${this.key}.backup`]) {
      try {
        const encoded = this.backend.getItem(key);
        if (encoded === null || encoded === '') continue;
        found = true;
        valid.push(this.decode(key, encoded));
      } catch { warning = true; }
    }
    valid.sort((a, b) => b.revision - a.revision || Number(a.key.endsWith('.backup')) -
Number(b.key.endsWith('.backup')));
    return { valid, warning, found };
  }
  load(): SnapshotRead<T> {
    const inspected = this.inspect();
    const latest = inspected.valid[0];
    this.revision = latest?.revision ?? 0;
    this.error = inspected.warning ? '部分本地存档不可用，已尝试恢复有效副本' : '';
    return { value: latest?.value ?? null, revision: this.revision,
      warning: inspected.warning, found: inspected.found };
  }
  write(value: T): boolean {
    try {
      const inspected = this.inspect();
      const nextRevision = Math.max(this.revision, ...inspected.valid.map(snapshot =>
snapshot.revision)) + 1;
      if (!Number.isSafeInteger(nextRevision)) throw new Error('Snapshot revision exhausted');
      const encoded = JSON.stringify({ snapshotFormat: 1, revision: nextRevision, payload: value });
      this.decode(this.key, encoded); // Never overwrite good storage with unserialisable/invalid
state.
      const latestKey = inspected.valid[0]?.key;
      // Write the older, invalid or absent slot first, preserving the newest legal previous save.
      const firstKey = latestKey === this.key ? `${this.key}.backup` : this.key;
      const secondKey = firstKey === this.key ? `${this.key}.backup` : this.key;
      let committed = 0;
      for (const key of [firstKey, secondKey]) {
        if (key === latestKey && !committed) continue;
        try {
          this.backend.setItem(key, encoded);
          const readback = this.backend.getItem(key);
          if (readback !== encoded) throw new Error('Snapshot readback mismatch');
          this.decode(key, readback);
          committed++;
        } catch {
          // Keep the newest old copy if no replacement has been verified. With empty storage,
          // trying the other slot is safe; after a verified write, mirroring is also safe.
        }
```

```
      }
      if (!committed) throw new Error('No new snapshot verified');
      this.revision = nextRevision;
      this.error = committed < 2 ? '进度已保存，备用存档暂时不可写' : '';
      return true;
    } catch {
      this.error = '本地空间不足或存储不可用，本次进度尚未保存';
      return false;
    }
  }
}
export type VideoResult = 'completed' | 'cancelled' | 'failed';
export type PresentVideoSimulation = (finish: () => void, cancel: () => void, fail: () => void) =>
void;
export interface VideoCloseResult { isEnded?: boolean; count?: number; }
export interface RewardedVideoHandle {
  load(): Promise<unknown>;
  show(): Promise<unknown>;
  onClose(callback: (result?: VideoCloseResult) => void): void;
  offClose?(callback: (result?: VideoCloseResult) => void): void;
  onError(callback: (error: unknown) => void): void;
  offError?(callback: (error: unknown) => void): void;
  destroy?(): void;
}
export interface DouyinVideoRuntime {
  createRewardedVideoAd?(options: { adUnitId: string }): RewardedVideoHandle;
}
export interface RewardedVideoOptions {
  /** Actual platform-issued ad unit, configured outside game rules. Empty means unavailable. */
  adUnitId?: string;
  /** Tests may inject a runtime; null explicitly selects the labelled web simulation. */
  runtime?: DouyinVideoRuntime | null;
  /** Upper bound for a callback that never arrives; does not award anything. */
  timeoutMs?: number;
}
function globalRuntime(): DouyinVideoRuntime | null {
  const runtime = (globalThis as unknown as { tt?: DouyinVideoRuntime }).tt;
  return runtime === undefined ? null : runtime ?? {};
}
export function isDouyinRuntime(): boolean {
  return typeof (globalThis as unknown as { tt?: unknown }).tt !== 'undefined';
}
/** Full viewing is confirmed by the real SDK close result; web simulation must be explicit. */
export class RewardedVideo {
  readonly simulationAvailable: boolean;
  readonly available: boolean;
  readonly unavailableReason: string;
  private readonly runtime: DouyinVideoRuntime | null;
  private readonly adUnitId: string;
  private readonly timeoutMs: number;
  private cancelActive: (() => void) | null = null;
```

```
  constructor(options: RewardedVideoOptions = {}) {
    this.runtime = options.runtime === undefined ? globalRuntime() : options.runtime;
    this.adUnitId = options.adUnitId?.trim() ?? '';
    this.timeoutMs = Math.max(1000, options.timeoutMs ?? 180000);
    this.simulationAvailable = this.runtime === null;
    this.available = this.simulationAvailable || (this.adUnitId.length > 0 && typeof
this.runtime?.createRewardedVideoAd === 'function');
    this.unavailableReason = !this.adUnitId ? '激励视频尚未配置，请免费重试' : '当前环境暂不支持激励视频，请
免费重试';
  }
  get isShowing(): boolean { return this.cancelActive !== null; }
  show(onResult: (outcome: VideoResult) => void, presentSimulation?: PresentVideoSimulation): void {
    if (this.cancelActive || !this.available) { onResult('failed'); return; }
    if (this.simulationAvailable) {
      if (!presentSimulation) { onResult('failed'); return; }
      let settled = false;
      const settle = (result: VideoResult) => {
        if (settled) return;
        settled = true;
        this.cancelActive = null;
        onResult(result);
      };
      this.cancelActive = () => settle('cancelled');
      try { presentSimulation(() => settle('completed'), () => settle('cancelled'), () =>
settle('failed')); }
      catch { settle('failed'); }
      return;
    }
    let ad: RewardedVideoHandle;
    try { ad = this.runtime!.createRewardedVideoAd!({ adUnitId: this.adUnitId }); }
    catch { onResult('failed'); return; }
    let settled = false;
    let timer: ReturnType<typeof setTimeout> | null = null;
    const close = (result?: VideoCloseResult) => {
      // Current SDKs return count even in ordinary single-video mode; follow its documented
      // precedence. One request still grants one reward, regardless of a larger count.
      const completed = result?.count !== undefined
        ? Number.isInteger(result.count) && result.count >= 1 : result?.isEnded === true;
      settle(completed ? 'completed' : 'cancelled');
    };
    const error = (_error: unknown) => settle('failed');
    const settle = (result: VideoResult) => {
      if (settled) return;
      settled = true;
      if (timer !== null) clearTimeout(timer);
      this.cancelActive = null;
      try { ad.offClose?.(close); } catch { /* SDK cleanup must not duplicate or suppress
settlement. */ }
      try { ad.offError?.(error); } catch { /* Same as above. */ }
      try { ad.destroy?.(); } catch { /* Same as above. */ }
      onResult(result);
    };
    this.cancelActive = () => settle('cancelled');
    try {
```

```
      ad.onClose(close);
      if (settled) return;
      ad.onError(error);
      if (settled) return;
      timer = setTimeout(() => settle('failed'), this.timeoutMs);
      Promise.resolve(ad.load()).then(() => {
        if (!settled) return ad.show();
        return undefined;
      }).catch(error);
    } catch (caught) { error(caught); }
  }
  /** Intentionally request only the explicitly labelled web simulation. */
  showSimulation(onResult: (outcome: VideoResult) => void, present: PresentVideoSimulation): void {
    if (!this.simulationAvailable) { onResult('failed'); return; }
    this.show(onResult, present);
  }
  dispose(): void { this.cancelActive?.(); }
}
import { AudioClip, AudioSource, Node, resources } from 'cc';
/** Optional, quiet feedback; missing audio or vibration never blocks an action. */
export class Feedback {
    private source: AudioSource;
    private clips: Partial<Record<'place' | 'complete', AudioClip>> = {};
    constructor(node: Node) {
        this.source = node.addComponent(AudioSource);
        for (const name of ['place', 'complete'] as const) {
            resources.load(`audio/${name}`, AudioClip, (error, clip) => {
                if (!error && clip) this.clips[name] = clip;
            });
        }
    }
    play(name: 'place' | 'complete', sound: boolean, haptics: boolean): void {
        const clip = this.clips[name];
        if (sound && clip) {
            try { this.source.playOneShot(clip, name === 'place' ? 0.3 : 0.45); } catch { /*
Optional capability. */ }
        }
        if (haptics) {
            const tt = (globalThis as unknown as { tt?: { vibrateShort?: (args: object) =>
void } }).tt;
            try { tt?.vibrateShort?.({ type: 'light', fail: () => {} }); } catch { /* Optional
capability. */ }
        }
    }
}
import { sys } from 'cc';
/** Browser accessibility for the Cocos canvas; the DOM layer never runs in a tt host. */
export class BrowserTestBridge {
    private controls: Record<string, { label: string; invoke: () => void }> = {};
    private texts: string[] = [];
    private sequence = 0;
    private layer: HTMLDivElement | null = null;
    constructor(enabled: boolean, private snapshot: () => object, private height = 844) {
```

```
        // Read-only diagnostic snapshot for the official IDE. It exposes no solution or commands.
        // A deep copy prevents external inspection from modifying the live wallet or attempt.
        try { Object.defineProperty(globalThis, '__SHISE_STATE__', {
            configurable: true, get: () => JSON.parse(JSON.stringify(this.snapshot())),
        }); } catch { /* Diagnostics never block a host without configurable global properties. */ }
        // Mini-game adapters also expose document, but it is not a browser DOM.
        if (!enabled || !sys.isBrowser || typeof document === 'undefined') return;
        this.layer = document.createElement('div');
        this.layer.setAttribute('aria-label', '拾色之间游戏操作');
        this.layer.style.cssText = 'position:fixed;inset:0;pointer-events:none;z-index:10';
        document.body.appendChild(this.layer);
        const style = document.createElement('style');
        style.textContent = '.shise-control{opacity:0;border:0;padding:0;pointer-
events:auto;position:fixed;cursor:pointer}.shise-control:focus-
visible{opacity:1;background:transparent;color:transparent;outline:2px solid #554D72;border-
radius:8px}';
        document.head.appendChild(style);
        const context = (document as unknown as { modelContext?: { registerTool: (tool: object) =>
Promise<unknown> } }).modelContext
            || (navigator as unknown as { modelContext?: { registerTool: (tool: object) =>
Promise<unknown> } }).modelContext;
        if (!context) return;
        const result = () => {
            try { return { content: [{ type: 'text', text: JSON.stringify({ ...this.snapshot(),
texts: [...this.texts],
                controls: Object.keys(this.controls).map(id => ({ id, label: this.controls[id].label
})) }) }] }; }
            catch (error) { return { content: [{ type: 'text', text: String(error) }] }; }
        };
        const register = (tool: object) => {
            try { Promise.resolve(context.registerTool(tool)).catch(() => {}); } catch { /* Optional
browser support. */ }
        };
        register({ name: 'shise_state', description: 'Read the visible Cocos tester UI state. Does
not reveal puzzle answers.',
            inputSchema: { type: 'object', properties: {} }, annotations: { readOnlyHint: true },
execute: result });
        register({ name: 'shise_tap', description: 'Tap a currently visible Cocos control by id from
shise_state. Same callback as screen controls; no arbitrary state edits.',
            inputSchema: { type: 'object', properties: { id: { type: 'string' } }, required:
['id'] },
            execute: (input: { id: string }) => { this.controls[input.id]?.invoke(); return
result(); } });
    }
    clear(): void { this.controls = {}; this.texts = []; this.layer?.replaceChildren(); }
    removeControls(labels: readonly string[]): void {
        for (const id of Object.keys(this.controls)) if (labels.includes(this.controls[id].label))
delete this.controls[id];
        if (this.layer) for (const button of Array.from(this.layer.querySelectorAll('button')))
            if (labels.includes(button.getAttribute('aria-label') || '')) button.remove();
    }
    modal(): void { this.controls = {}; this.layer?.replaceChildren(); }
    text(value: string): void { this.texts.push(value); }
    control(label: string, invoke: () => void, bounds?: { x: number; y: number; width: number;
height: number }): void {
        this.controls[`control-${++this.sequence}`] = { label, invoke };
        if (!this.layer || !bounds) return;
        const canvas = document.getElementById('GameCanvas');
        if (!canvas) return;
        const rect = canvas.getBoundingClientRect();
        const scale = Math.min(rect.width / 390, rect.height / this.height);
        const button = document.createElement('button');
        button.type = 'button'; button.className = 'shise-control';
        button.textContent = label; button.setAttribute('aria-label', label);
        button.style.left = `${rect.x + rect.width / 2 + (bounds.x - 195 - bounds.width / 2) *
scale}px`;
```

```
        button.style.top = `${rect.y + rect.height / 2 + (bounds.y - this.height / 2 - bounds.height
/ 2) * scale}px`;
        button.style.width = `${bounds.width * scale}px`; button.style.height = `${bounds.height *
scale}px`;
        button.addEventListener('click', invoke); this.layer.appendChild(button);
    }
}
/** Pure layout in the 390-wide design space; every tray home keeps its original page/slot. */
export function boardLayout(height: number, safeTop: number, safeBottom: number,
    columns: number, rows: number, trayCount: number) {
    const gap = 6;
    const reservedBoardTop = safeTop + 92;
    const propsY = height - safeBottom - 38;
    const actionsY = propsY - 72;
    // Footer buttons remain in a predictable location; the puzzle is a single centered group above
them.
    const groupLimitBottom = actionsY - 32;
    const trayColumns = Math.min(6, Math.max(1, trayCount));
    let trayRows = Math.min(3, Math.max(1, Math.ceil(trayCount / trayColumns)));
    const trayGap = rows >= 5 ? 32 : 44;
    const maxCellSize = columns <= 4 && rows <= 2 ? 68 : 52;
    const boardSize = (visibleTrayRows: number) => {
        const trayHeight = visibleTrayRows * 44 + (visibleTrayRows - 1) * gap;
        const hasPages = trayCount > trayColumns * visibleTrayRows;
        const available = groupLimitBottom - reservedBoardTop - trayHeight - trayGap - (hasPages ?
44 : 0);
        return Math.min(maxCellSize, (342 - (columns - 1) * gap) / columns,
            (available - (rows - 1) * gap) / rows);
    };
    while (trayRows > 1 && boardSize(trayRows) < 44) trayRows--;
    const cellSize = Math.max(24, boardSize(trayRows));
    const boardWidth = columns * cellSize + (columns - 1) * gap;
    const boardHeight = rows * cellSize + (rows - 1) * gap;
    const trayHeight = trayRows * 44 + (trayRows - 1) * gap;
    const trayWidth = trayColumns * 44 + (trayColumns - 1) * gap;
    const slotsPerPage = trayColumns * trayRows;
    const hasPages = trayCount > slotsPerPage;
    const groupHeight = boardHeight + trayGap + trayHeight;
    const completeHeight = groupHeight + (hasPages ? 44 : 0);
    const groupTop = reservedBoardTop + Math.max(0,
        (groupLimitBottom - reservedBoardTop - completeHeight) / 2);
    const trayTop = groupTop + boardHeight + trayGap;
    return {
        cellSize, pitch: cellSize + gap, boardTop: groupTop, boardHeight, boardWidth,
        boardStartX: 195 - (columns - 1) * (cellSize + gap) / 2,
        boardStartY: groupTop + cellSize / 2,
        trayColumns, trayRows, trayCellSize: 44, trayPitch: 50,
        trayTop, trayHeight, trayWidth, trayGap, trayLabelY: trayTop - 18,
        slotsPerPage, propsY, actionsY, pagerY: trayTop + trayHeight + 24,
        groupTop, groupHeight, groupWidth: Math.max(boardWidth, trayWidth),
        compact: cellSize < 44,
    };
}
import { BlockInputEvents, Color, Graphics, Label, Layers, Node, Sprite, SpriteFrame, UITransform }
from 'cc';
```

```
/** Keep these words as readable engine text, never baked into an illustration. */
export const HEALTHY_PLAY_LINES = [
  '抵制不良游戏，拒绝盗版游戏。',
  '注意自我保护，谨防受骗上当。',
  '适度游戏益脑，沉迷游戏伤身。',
  '合理安排时间，享受健康生活。',
] as const;
export const AGE_RATING_NOTE = '适龄提示 · 8+';
/** Kept as a source-compatible alias for callers; the displayed wording is the confirmed 8+ rating.
*/
export const DEVELOPMENT_AGE_NOTE = AGE_RATING_NOTE;
export interface LoadingScreenOptions {
  parent: Node;
  width: number;
  height: number;
  safeTop: number;
  safeBottom: number;
  /** An illustration only: the logo, progress and declarations are live native UI. */
  artwork?: SpriteFrame | null;
  onAgeInfo?: () => void;
  /** Optional browser-test mirror. The Cocos controls always remain authoritative. */
  registerControl?: (label: string, invoke: () => void,
    bounds: { x: number; y: number; width: number; height: number }) => void;
  /** Remove only the supplied module-owned labels; never clear other screen controls. */
  clearControls?: (labels: readonly string[]) => void;
  onModal?: (open: boolean) => void;
}
/** A native Cocos loading view. Loading completion is controlled by its caller. */
export class LoadingScreen {
  readonly node: Node;
  private readonly options: LoadingScreenOptions;
  private readonly artworkNode: Node;
  private readonly fallbackNode: Node;
  private readonly fill: Graphics;
  private readonly status: Label;
  private readonly percentage: Label;
  private readonly trackWidth: number;
  private ageLayer: Node | null = null;
  private progress = 0;
  private controlLabels = new Set<string>();
  private destroying = false;
  constructor(options: LoadingScreenOptions) {
    this.options = options;
    const { parent, width, height, safeTop, safeBottom } = options;
    this.node = this.element('LoadingScreen', width / 2, height / 2, width, height, parent);
    // The parent uses the same center-origin UI space as Boot's root.
    this.node.setPosition(0, 0);
    this.node.addComponent(BlockInputEvents);
    this.round('CreamBackdrop', width / 2, height / 2, width + 2, height + 2, 0, '#FFF8EE');
    this.fallbackNode = this.element('ArtworkFallback', width / 2, height / 2, width, height);
    this.fallbackNode.setPosition(0, 0);
```

```
    this.drawFallback();
    this.artworkNode = this.element('LoadingIllustration', width / 2, height / 2, width, height);
    this.artworkNode.addComponent(Sprite).sizeMode = Sprite.SizeMode.CUSTOM;
    this.artworkNode.active = false;
    if (options.artwork) this.setArtwork(options.artwork);
    // The user has confirmed 8+ with Douyin. Keep the mark readable above the illustration.
    const badgeY = safeTop + 31;
    const ageBadge = this.round('Age8PlusBadge', 40, badgeY, 48, 54, 4, '#65AF32');
    this.label('适龄提示', 40, badgeY - 17, 46, 11, 8, '#FFFFFF', true);
    this.label('8+', 40, badgeY + 4, 45, 38, 30, '#FFFFFF', true);
    ageBadge.on(Node.EventType.TOUCH_END, () => this.invokeAgeInfo());
    const titleY = Math.max(54, safeTop + 38);
    this.label('拾色之间', width / 2, titleY, width - 48, 58, 38, '#564740', true);
    this.label('收集自然的每一种颜色', width / 2, titleY + 42, width - 56, 28, 14, '#746053');
    const panelTop = height - safeBottom - 242;
    this.round('LoadingInformation', width / 2, (panelTop + height) / 2 + 16,
      width + 2, height - panelTop + 32, 36, '#FFF8EE');
    this.status = this.label('正在准备颜色', width / 2, panelTop + 28, width - 66, 26, 14, '#68574B');
    this.trackWidth = Math.min(288, width - 82);
    this.round('ProgressTrack', width / 2, panelTop + 58, this.trackWidth, 10, 5, '#EADDD0');
    const fillNode = this.element('ProgressFill', width / 2, panelTop + 58, this.trackWidth, 10);
    this.fill = fillNode.addComponent(Graphics);
    this.percentage = this.label('0%', width / 2, panelTop + 82, width - 72, 22, 12, '#756356');
    this.label('健康游戏忠告', width / 2, panelTop + 111, width - 48, 20, 12, '#655345', true);
    HEALTHY_PLAY_LINES.forEach((line, i) => {
      this.label(line, width / 2, panelTop + 134 + i * 18, width - 36, 18, 12, '#6E6055');
    });
    const ageEntry = this.round('AgeInformationEntry', width / 2, panelTop + 219,
      Math.min(244, width - 60), 34, 17, '#F1E7DA');
    ageEntry.getComponent(UITransform)!.setContentSize(Math.min(244, width - 60), 44);
    const ageLabel = this.label(DEVELOPMENT_AGE_NOTE, width / 2, panelTop + 219,
      width - 72, 30, 12, '#655549');
    // The label sits above the button but has no listener and does not intercept touches.
    ageEntry.on(Node.EventType.TOUCH_END, () => this.invokeAgeInfo());
    ageLabel.name = 'AgeInformationLabel';
    this.registerAgeEntry();
    this.setProgress(0);
  }
  /** Updates actual resource progress; it never advances on a fake timer. */
  setProgress(progress: number, status?: string): void {
    if (!this.node.isValid) return;
    this.progress = Math.max(this.progress, Math.min(1, Number.isFinite(progress) ? progress : 0));
    if (status !== undefined) this.status.string = status;
    this.percentage.string = `${Math.round(this.progress * 100)}%`;
    this.fill.clear();
    const fillWidth = this.trackWidth * this.progress;
    if (fillWidth <= 0) return;
    this.fill.fillColor = new Color().fromHEX('#EA987B');
    this.fill.roundRect(-this.trackWidth / 2, -5, fillWidth, 10, Math.min(5, fillWidth / 2));
    this.fill.fill();
```

```
  }
  /** Cover the viewport without distorting the source illustration. */
  setArtwork(frame: SpriteFrame): void {
    if (!this.node.isValid) return;
    const { width, height } = this.options;
    const source = frame.originalSize;
    const scale = Math.max(width / Math.max(1, source.width), height / Math.max(1, source.height));
    const sprite = this.artworkNode.getComponent(Sprite)!;
    sprite.sizeMode = Sprite.SizeMode.CUSTOM;
    sprite.spriteFrame = frame;
    this.artworkNode.getComponent(UITransform)!.setContentSize(source.width * scale, source.height *
scale);
    this.artworkNode.active = true;
    this.fallbackNode.active = false;
  }
  /** Confirmed age guidance, available from both the 8+ badge and footer entry. */
  openAgeInfo(): void {
    if (!this.node.isValid || this.ageLayer?.isValid) return;
    this.clearOwnControls();
    this.options.onModal?.(true);
    const { width, height } = this.options;
    this.ageLayer = this.round('AgeInformationScrim', width / 2, height / 2,
      width + 2, height + 2, 0, '#594E48');
    this.ageLayer.getComponent(Graphics)!.fillColor = new Color(89, 78, 72, 92);
    const scrim = this.ageLayer.getComponent(Graphics)!;
    scrim.clear(); scrim.rect(-width / 2, -height / 2, width, height); scrim.fill();
    this.ageLayer.addComponent(BlockInputEvents);
    const centerY = height / 2;
    this.round('AgeInformationCard', width / 2, centerY, width - 44, 358, 28, '#FFF9F0',
this.ageLayer);
    this.label('适龄说明 · 8+', width / 2, centerY - 132, width - 84, 36, 22, '#564740', true,
this.ageLayer);
    this.label('适合年满 8 周岁及以上用户', width / 2, centerY - 91, width - 84, 26, 14, '#65AF32',
true, this.ageLayer);
    this.label('本游戏为休闲益智类色彩拼图，\n 通过点选色块补全渐变，包含倒计时挑战、\n 积分兑换道具和自愿观看激励视
频功能。',
      width / 2, centerY - 29, width - 76, 78, 13, '#6E5D50', false, this.ageLayer);
    this.label('游戏无陌生人社交及现金充值入口。\n 建议未成年人在监护人指导下体验，\n 合理安排游戏时间，避免长时间连
续游戏。',
      width / 2, centerY + 58, width - 76, 78, 13, '#6E5D50', false, this.ageLayer);
    const close = this.round('CloseAgeInformation', width / 2, centerY + 137,
      168, 42, 21, '#F3CEB2', this.ageLayer);
    this.label('知道了', width / 2, centerY + 137, 152, 38, 15, '#564740', true, this.ageLayer);
    close.on(Node.EventType.TOUCH_END, () => this.closeAgeInfo());
    this.register('关闭适龄说明', () => this.closeAgeInfo(),
      { x: width / 2, y: centerY + 137, width: 168, height: 42 });
  }
  closeAgeInfo(): void {
    if (!this.ageLayer) return;
    this.clearOwnControls();
    this.ageLayer.removeFromParent(); this.ageLayer.destroy(); this.ageLayer = null;
    if (!this.destroying) {
      this.options.onModal?.(false);
      this.registerAgeEntry();
    }
  }
```

```
  get isAgeInfoOpen(): boolean { return !!this.ageLayer?.isValid; }
  destroy(): void {
    this.destroying = true;
    this.closeAgeInfo();
    this.clearOwnControls();
    if (!this.node.isValid) return;
    this.node.removeFromParent(); this.node.destroy();
  }
  private invokeAgeInfo(): void {
    if (this.options.onAgeInfo) this.options.onAgeInfo();
    else this.openAgeInfo();
  }
  private registerAgeEntry(): void {
    const { width, height, safeTop, safeBottom } = this.options;
    this.register('8+ 适龄提示', () => this.invokeAgeInfo(),
      { x: 40, y: safeTop + 31, width: 48, height: 54 });
    this.register(AGE_RATING_NOTE, () => this.invokeAgeInfo(),
      { x: width / 2, y: height - safeBottom - 23, width: Math.min(244, width - 60), height: 44 });
  }
  private register(label: string, invoke: () => void,
    bounds: { x: number; y: number; width: number; height: number }): void {
    this.controlLabels.add(label);
    this.options.registerControl?.(label, invoke, bounds);
  }
  private clearOwnControls(): void {
    if (this.controlLabels.size) this.options.clearControls?.([...this.controlLabels]);
    this.controlLabels.clear();
  }
  private element(name: string, x: number, y: number, width: number, height: number,
    parent: Node = this.node): Node {
    const node = new Node(name);
    node.layer = Layers.Enum.UI_2D;
    node.addComponent(UITransform).setContentSize(width, height);
    node.setPosition(x - this.options.width / 2, this.options.height / 2 - y);
    parent.addChild(node);
    return node;
  }
  private round(name: string, x: number, y: number, width: number, height: number,
    radius: number, hex: string, parent: Node = this.node): Node {
    const node = this.element(name, x, y, width, height, parent);
    // Containers share the screen's center origin; nested visual children still use absolute UI
coordinates.
    if (parent !== this.node) {
      const parentPosition = parent.position;
      node.setPosition(node.position.x - parentPosition.x, node.position.y - parentPosition.y);
    }
    const graphics = node.addComponent(Graphics);
    graphics.fillColor = new Color().fromHEX(hex);
    graphics.roundRect(-width / 2, -height / 2, width, height, Math.min(radius, width / 2, height /
2));
    graphics.fill();
    return node;
```

```
  }
  private label(value: string, x: number, y: number, width: number, height: number,
    size: number, hex: string, bold = false, parent: Node = this.node): Label {
    const node = this.element('LoadingText', x, y, width, height, parent);
    if (parent !== this.node) {
      const parentPosition = parent.position;
      node.setPosition(node.position.x - parentPosition.x, node.position.y - parentPosition.y);
    }
    const label = node.addComponent(Label);
    label.string = value;
    label.useSystemFont = true;
    label.fontFamily = 'sans-serif';
    label.fontSize = size;
    label.lineHeight = size * 1.45;
    label.isBold = bold;
    label.color = new Color().fromHEX(hex);
    label.horizontalAlign = Label.HorizontalAlign.CENTER;
    label.verticalAlign = Label.VerticalAlign.CENTER;
    label.enableWrapText = true;
    label.overflow = Label.Overflow.NONE;
    return label;
  }
  private drawFallback(): void {
    const { width, height, safeBottom } = this.options;
    const heroY = Math.min(height * 0.45, height - safeBottom - 338);
    this.round('SoftSky', width / 2, height * 0.28, width + 4, height * 0.58, 0, '#F7E7DF',
this.fallbackNode);
    this.round('CloudLeft', 17, heroY - 89, 172, 82, 40, '#FFF9F0', this.fallbackNode);
    this.round('CloudRight', width - 12, heroY - 144, 196, 96, 48, '#FFF6EB', this.fallbackNode);
    this.round('MintHill', width / 2 + 68, heroY + 124, width + 90, 204, 102, '#DAE9D6',
this.fallbackNode);
    this.round('PeachHill', 43, heroY + 174, width + 76, 174, 86, '#F3C7AC', this.fallbackNode);
    const colors = ['#F0AC86', '#E5A4AB', '#B4ABC9', '#92BCB1', '#E7CC8E'];
    colors.forEach((hex, i) => {
      const tile = this.round('FloatingColor', width / 2 + (i - 2) * 48,
        heroY - 4 + Math.abs(i - 2) * 18, 40, 40, 11, hex, this.fallbackNode);
      tile.setRotationFromEuler(0, 0, (i - 2) * -7);
    });
  }
}
import { Color, EventTouch, Graphics, Label, Layers, Node, Tween, tween, UITransform } from 'cc';
export type TutorialControlBounds = { x: number; y: number; width: number; height: number };
export type TutorialOptions = {
    /** The demonstration never reads or writes a real level, wallet, or save. */
    onStart?: () => void;
    registerControl?: (label: string, invoke: () => void, bounds: TutorialControlBounds) => void;
    clearControls?: () => void;
    onText?: (text: string) => void;
    reducedMotion?: boolean;
};
const palette = {
    background: '#FFF9F1', ink: '#615147', muted: '#918177', white: '#FFFFFF',
```

```
    coral: '#EE8B75', mint: '#DBEFE6', lilac: '#EEE7F6', butter: '#FFF0C8',
    border: '#EDE3D9', line: '#C8D8CE', selection: '#7E9C8F',
};
/** Three looping native-Cocos demonstrations. No real gameplay state is involved. */
export class TutorialScreen {
    private root: Node;
    private page = 0;
    private disposed = false;
    private animationClock: { elapsed: number } | null = null;
    constructor(parent: Node, private height: number, private top: number, private bottom: number,
        private onExit: () => void, private options: TutorialOptions = {}) {
        this.root = new Node('TutorialScreen');
        this.root.layer = Layers.Enum.UI_2D;
        this.root.addComponent(UITransform).setContentSize(390, height);
        parent.addChild(this.root);
        this.render();
    }
    dispose(): void {
        if (this.disposed) return;
        this.disposed = true;
        this.stopAnimation();
        this.options.clearControls?.();
        this.root.removeFromParent();
        this.root.destroy();
    }
    private stopAnimation(): void {
        if (this.animationClock) Tween.stopAllByTarget(this.animationClock);
        this.animationClock = null;
    }
    private render(): void {
        if (this.disposed) return;
        this.stopAnimation();
        this.options.clearControls?.();
        for (const child of [...this.root.children]) { child.removeFromParent(); child.destroy(); }
        this.box(195, this.height / 2, 390, this.height, 0, palette.background);
        this.circle(340, this.top + 143, 67, '#FFF0E6');
        this.circle(37, this.height - this.bottom - 142, 61, '#F2F5E9');
        this.label('拾色小课堂', 195, this.top + 26, 240, 36, 23, palette.ink, true);
        this.iconButton('退出教程', 38, this.top + 26, () => this.onExit());
        for (let i = 0; i < 3; i++) {
            this.box(166 + i * 29, this.top + 71, i === this.page ? 26 : 8, 8, 4,
                i === this.page ? palette.coral : '#E8DFD3');
        }
        const titles = ['点选颜色，补全渐变', '轻点改放，轻点收回', '三个帮手，从容拾色'];
        const descriptions = ['点下方颜色，再点上方空位', '改放到空位 · 收回原来的候选槽', '通关得积分，在道具屋
兑换'];
        this.label(titles[this.page], 195, this.top + 115, 342, 35, 22, palette.ink, true);
        this.label(descriptions[this.page], 195, this.top + 153, 342, 30, 14, palette.muted);
        const cardTop = this.top + 185;
        const cardBottom = this.height - this.bottom - 128;
        this.box(195, (cardTop + cardBottom) / 2, 342, cardBottom - cardTop, 28, palette.white);
```

```
        const boardY = cardTop + Math.min(84, (cardBottom - cardTop) * 0.24);
        const trayY = boardY + 123;
        if (this.page === 0) this.fillDemo(boardY, trayY, cardBottom);
        else if (this.page === 1) this.adjustDemo(boardY, trayY, cardBottom);
        else this.propsDemo(cardTop, cardBottom);
        this.footer();
    }
    private loop(duration: number, draw: (elapsed: number) => void): void {
        const clock = { elapsed: 0 };
        this.animationClock = clock;
        draw(0);
        // The only repeating work is owned by this clock and is stopped on every page
change/dispose.
        tween(clock).to(duration, { elapsed: duration }, { onUpdate: () => {
            if (!this.disposed && this.root.isValid) draw(clock.elapsed);
        } }).call(() => { clock.elapsed = 0; draw(0); }).union().repeatForever().start();
    }
    private fillDemo(boardY: number, trayY: number, cardBottom: number): void {
        this.track([121, 195, 269], boardY);
        this.box(121, boardY, 66, 66, 12, '#DF6267');
        this.box(195, boardY, 66, 66, 12, '#FBF9F5', palette.border);
        this.box(269, boardY, 66, 66, 12, '#FFC18B');
        this.pin(121, boardY - 39); this.pin(269, boardY - 39);
        this.box(195, trayY, 100, 82, 24, '#F5F6EF');
        this.box(195, trayY, 58, 58, 12, '#ECEFE6');
        const selection = this.box(195, trayY, 76, 76, 17, palette.mint);
        const target = this.box(195, boardY, 76, 76, 17, palette.mint);
        // The moving copy is used only to explain a click; gameplay input itself remains tap-to-
fill.
        const tile = this.box(195, trayY, 66, 66, 12, '#EF8C74');
        const hand = this.hand(211, trayY + 44);
        const hint = this.label('', 195, cardBottom - 29, 300, 30, 14, palette.selection, true);
        this.loop(5.6, t => {
            const flight = this.fraction(t, 2.0, 2.6);
            this.display(selection, t >= 0.6 && t < 2.0);
            this.display(target, t >= 1.5 && t < 2.7);
            this.place(tile, 195, trayY + (boardY - trayY) * (this.options.reducedMotion ? (t >= 2.3
? 1 : 0) : flight));
            const handTravel = this.fraction(t, 0.9, 1.5);
            this.place(hand, 211, trayY + 44 + (boardY - trayY) * handTravel);
            this.display(hand, t < 2.7 && !this.options.reducedMotion, this.tapScale(t, [0.65,
1.75]));
            hint.string = t < 1.3 ? '① 点选颜色' : t < 2.7 ? '② 点上方空位' : '渐变完成';
        });
    }
    private adjustDemo(boardY: number, trayY: number, cardBottom: number): void {
        const xs = [92, 160, 228, 296];
        this.track(xs, boardY);
        xs.forEach((x, i) => this.box(x, boardY, 58, 58, 12,
            i === 0 ? '#65A0AB' : i === 3 ? '#E3EDCA' : '#FBF9F5', i === 1 || i === 2 ?
palette.border : undefined));
        this.pin(xs[0], boardY - 35); this.pin(xs[3], boardY - 35);
        this.box(195, trayY, 100, 82, 24, '#F5F6EF');
        this.box(195, trayY, 58, 58, 12, '#ECEFE6', '#D4DECE');
        this.returnIcon(195, trayY, '#A2B1A3');
```

```
        const selection = this.box(160, boardY, 68, 68, 17, palette.mint);
        const destination = this.box(228, boardY, 68, 68, 17, palette.mint);
        const tile = this.box(160, boardY, 58, 58, 12, '#B9DCC8');
        const hand = this.hand(174, boardY + 41);
        const hint = this.label('', 195, cardBottom - 29, 310, 30, 14, palette.selection, true);
        this.loop(5.8, t => {
            const moving = this.fraction(t, 1.25, 1.8);
            const returning = this.fraction(t, 3.35, 4.0);
            let tileX = 160 + 68 * moving;
            let tileY = boardY;
            if (t >= 3.35) { tileX = 228 + (195 - 228) * returning; tileY += (trayY - boardY) *
returning; }
            if (this.options.reducedMotion) {
                tileX = t < 1.55 ? 160 : t < 3.7 ? 228 : 195;
                tileY = t < 3.7 ? boardY : trayY;
            }
            this.place(tile, tileX, tileY);
            this.place(selection, t < 2.5 ? 160 : 228, boardY);
            this.display(selection, (t > 0.4 && t < 1.25) || (t > 2.5 && t < 3.35));
            this.place(destination, t < 2.5 ? 228 : 195, t < 2.5 ? boardY : trayY);
            this.display(destination, (t > 0.9 && t < 1.8) || (t > 3.0 && t < 4.0));
            let handX = t < 0.8 ? 174 : t < 2.5 ? 174 + 68 * this.fraction(t, 0.8, 1.2) : 242;
            let handY = boardY + 41;
            if (t >= 2.9) { const a = this.fraction(t, 2.9, 3.3); handX = 242 + (209 - 242) * a;
handY += (trayY - boardY) * a; }
            this.place(hand, handX, handY);
            this.display(hand, t < 4.1 && !this.options.reducedMotion, this.tapScale(t, [0.5, 1.2,
2.65, 3.35]));
            hint.string = t < 2.4 ? '改放：点色块，再点空位' : t < 4.1 ? '收回：点色块，再点任意空槽' : '已
收回待选区';
        });
    }
    private propsDemo(cardTop: number, cardBottom: number): void {
        const available = cardBottom - cardTop;
        const rowHeight = Math.min(88, (available - 36) / 3);
        const firstY = cardTop + 28 + rowHeight / 2;
        const rows = [
            { title: '点睛', detail: '自动填对并固定 1 块', color: palette.mint, icon: 'wand' },
            { title: '窥色', detail: '查看完整色带 3 秒', color: palette.lilac, icon: 'eye' },
            { title: '续时', detail: '本关倒计时增加 30 秒', color: palette.butter, icon: 'plus' },
        ] as const;
        const highlights: Node[] = [];
        rows.forEach((row, index) => {
            const y = firstY + index * rowHeight;
            highlights.push(this.box(195, y, 318, rowHeight - 5, 20, row.color));
            this.circle(72, y, 23, row.color); this.icon(row.icon, 72, y, palette.ink);
            this.label(row.title, 153, y - 13, 100, 28, 17, palette.ink, true);
            this.label(row.detail, 220, y + 13, 233, 27, 12, palette.muted);
        });
        const hand = this.hand(305, firstY + 15);
        const stars = [this.spark(300, firstY - 14, 7, '#DCA34B'),
            this.spark(313, firstY - 2, 4, '#DCA34B')];
        this.loop(5.7, t => {
            const row = Math.min(2, Math.floor(t / 1.9));
```

```
            const phase = (t % 1.9) / 1.9;
            highlights.forEach((node, i) => this.display(node, i === row, 1));
            const travel = this.options.reducedMotion ? row : Math.min(2, t / 1.9);
            this.place(hand, 305, firstY + Math.min(2, travel) * rowHeight + 15);
            this.display(hand, !this.options.reducedMotion, phase > 0.2 && phase < 0.42 ? 0.88 : 1);
            stars.forEach((star, i) => { this.place(star, 300 + i * 13, firstY + row * rowHeight -
14 + i * 12);
                this.display(star, phase > 0.42 && phase < 0.8, 0.7 + Math.sin(phase * Math.PI) *
0.4); });
        });
    }
    private fraction(t: number, start: number, end: number): number {
        const value = Math.max(0, Math.min(1, (t - start) / (end - start)));
        return value * value * (3 - 2 * value);
    }
    private tapScale(t: number, taps: number[]): number {
        if (this.options.reducedMotion) return 1;
        return taps.some(tap => Math.abs(t - tap) < 0.12) ? 0.87 : 1;
    }
    private place(node: Node, x: number, y: number): void { node.setPosition(x - 195, this.height /
2 - y); }
    private display(node: Node, visible: boolean, scale = 1): void {
        node.active = visible; node.setScale(scale, scale, 1);
    }
    private track(xs: number[], y: number): void {
        const g = this.shape(195, y); g.strokeColor = this.color(palette.line); g.lineWidth = 3;
        g.moveTo(xs[0] - 195, 0); g.lineTo(xs[xs.length - 1] - 195, 0); g.stroke();
    }
    private footer(): void {
        const y = this.height - this.bottom - 55;
        if (this.page > 0) this.button('上一步', 79, y, 108, 54, palette.white, palette.muted,
            () => { this.page--; this.render(); });
        this.button(this.page === 2 ? '开始拾色' : '下一步', this.page === 0 ? 195 : 252,
            y, this.page === 0 ? 276 : 214, 54, palette.coral, '#FFFFFF', () => {
                if (this.page === 2) (this.options.onStart ?? this.onExit)();
                else { this.page++; this.render(); }
            });
        this.label('自动循环演示 · 随时翻页', 195, this.height - this.bottom - 11, 342, 22, 11,
palette.muted);
    }
    private node(name: string, x: number, y: number, width = 1, height = 1): Node {
        const node = new Node(name); node.layer = Layers.Enum.UI_2D;
        node.addComponent(UITransform).setContentSize(width, height);
        node.setPosition(x - 195, this.height / 2 - y); this.root.addChild(node); return node;
    }
    private color(hex: string): Color { return new Color().fromHEX(hex); }
    private shape(x: number, y: number): Graphics { return this.node('TutorialShape', x,
y).addComponent(Graphics); }
    private box(x: number, y: number, width: number, height: number, radius: number, fill: string,
stroke?: string): Node {
        const g = this.shape(x, y); g.fillColor = this.color(fill);
        g.roundRect(-width / 2, -height / 2, width, height, Math.min(radius, height / 2)); g.fill();
        if (stroke) { g.strokeColor = this.color(stroke); g.lineWidth = 2; g.stroke(); }
        return g.node;
    }
    private circle(x: number, y: number, radius: number, fill: string): Node {
```

```
        const g = this.shape(x, y); g.fillColor = this.color(fill); g.circle(0, 0, radius);
g.fill(); return g.node;
    }
    private label(text: string, x: number, y: number, width: number, height: number, size: number,
        color = palette.ink, bold = false): Label {
        this.options.onText?.(text);
        const label = this.node('TutorialText', x, y, width, height).addComponent(Label);
        label.string = text; label.useSystemFont = true; label.fontFamily = 'sans-serif';
        label.fontSize = size; label.lineHeight = size * 1.45; label.isBold = bold; label.color =
this.color(color);
        label.horizontalAlign = Label.HorizontalAlign.CENTER; label.verticalAlign =
Label.VerticalAlign.CENTER;
        label.overflow = Label.Overflow.SHRINK; return label;
    }
    private hit(label: string, x: number, y: number, width: number, height: number, callback: () =>
void): void {
        const invoke = () => { if (!this.disposed) callback(); };
        this.node(label, x, y, width, height).on(Node.EventType.TOUCH_END, (event: EventTouch) => {
            event.propagationStopped = true; invoke();
        });
        this.options.registerControl?.(label, invoke, { x, y, width, height });
    }
    private button(label: string, x: number, y: number, width: number, height: number, fill: string,
        color: string, callback: () => void): void {
        this.box(x, y, width, height, height / 2, fill);
        this.label(label, x, y, width - 12, height - 4, 17, color, true);
        this.hit(label, x, y, width, height, callback);
    }
    private iconButton(label: string, x: number, y: number, callback: () => void): void {
        this.circle(x, y, 23, palette.white); this.icon('close', x, y, palette.muted);
        this.hit(label, x, y, 46, 46, callback);
    }
    private pin(x: number, y: number): void {
        this.circle(x, y, 5, palette.white); this.circle(x, y, 2.7, '#9CA69B');
    }
    private spark(x: number, y: number, radius: number, color: string): Node {
        const g = this.shape(x, y); g.fillColor = this.color(color);
        g.moveTo(0, radius); g.lineTo(radius * 0.28, radius * 0.28); g.lineTo(radius, 0);
        g.lineTo(radius * 0.28, -radius * 0.28); g.lineTo(0, -radius);
        g.lineTo(-radius * 0.28, -radius * 0.28); g.lineTo(-radius, 0);
        g.lineTo(-radius * 0.28, radius * 0.28); g.close(); g.fill(); return g.node;
    }
    private hand(x: number, y: number): Node {
        const g = this.shape(x, y); g.strokeColor = this.color(palette.coral); g.lineWidth = 2.5;
        g.fillColor = this.color('#FFF4E9');
        g.moveTo(-5, -15); g.lineTo(-12, -3); g.bezierCurveTo(-15, 2, -10, 5, -7, 1);
        g.lineTo(-5, -1); g.lineTo(-5, 15); g.bezierCurveTo(-5, 21, 1, 21, 1, 15);
        g.lineTo(1, 6); g.bezierCurveTo(13, 10, 17, 2, 12, -12);
        g.lineTo(8, -19); g.lineTo(-1, -19); g.close(); g.fill(); g.stroke(); return g.node;
    }
    private returnIcon(x: number, y: number, color: string): void {
        const g = this.shape(x, y); g.strokeColor = this.color(color); g.lineWidth = 2;
        g.moveTo(-9, 6); g.lineTo(3, 6); g.bezierCurveTo(12, 6, 12, -8, 3, -8);
        g.moveTo(-4, 12); g.lineTo(-10, 6); g.lineTo(-4, 0); g.stroke();
```

```
    }
    private icon(kind: 'close' | 'wand' | 'eye' | 'plus', x: number, y: number, color: string): void
{
        const g = this.shape(x, y); g.strokeColor = this.color(color); g.lineWidth = 2.6;
        if (kind === 'close') {
            g.moveTo(-6, -6); g.lineTo(6, 6); g.moveTo(-6, 6); g.lineTo(6, -6); g.stroke();
        } else if (kind === 'wand') {
            g.moveTo(-10, -11); g.lineTo(7, 8); g.stroke();
            this.spark(x + 8, y - 9, 6, color); this.spark(x - 5, y - 14, 3, color);
        } else if (kind === 'eye') {
            g.moveTo(-15, 0); g.bezierCurveTo(-6, 13, 6, 13, 15, 0);
            g.bezierCurveTo(6, -13, -6, -13, -15, 0); g.stroke();
            g.fillColor = this.color(color); g.circle(0, 0, 4.5); g.fill();
        } else {
            g.circle(0, 0, 13); g.moveTo(-6, 0); g.lineTo(6, 0); g.moveTo(0, -6); g.lineTo(0, 6);
g.stroke();
        }
    }
}
import {
  BlockInputEvents, Color, EventTouch, Graphics, Label, Layers, Node,
  Sprite, SpriteFrame, UITransform,
} from 'cc';
export interface WinPanelOptions {
  parent: Node;
  width: number;
  height: number;
  safeTop: number;
  safeBottom: number;
  score: number;
  pointsEarned: number;
  elapsedMs: number;
  movesSpent: number;
  levelNumber: number;
  levelTitle: string;
  /** True when this completion can still request the one-time extra reward. */
  doubleAvailable: boolean;
  onDouble: () => void;
  onNext: () => void;
  onHome: () => void;
  /** Transparent celebration artwork; fitted with contain and never stretched. */
  artwork?: SpriteFrame | null;
  registerControl?: (label: string, invoke: () => void,
    bounds: { x: number; y: number; width: number; height: number }) => void;
  onText?: (value: string) => void;
}
const C = {
  ink: '#655148', muted: '#9B8779', white: '#FFFEFB', coral: '#EC8D76',
  mint: '#EAF4E9', green: '#648770', gold: '#FFF0C4', goldInk: '#97702E',
};
/** A self-contained native victory composition, with no gameplay or reward mutations. */
export class WinPanel {
```

```
  readonly node: Node;
  private disposed = false;
  private readonly cardWidth: number;
  constructor(private readonly options: WinPanelOptions) {
    const { parent, width, height, safeTop, safeBottom } = options;
    this.node = new Node('WinPanel');
    this.node.layer = Layers.Enum.UI_2D;
    this.node.addComponent(UITransform).setContentSize(width, height);
    parent.addChild(this.node);
    this.node.setPosition(0, 0);
    this.node.addComponent(BlockInputEvents);
    const scrim = this.element('WinBackdrop', width / 2, height / 2, width + 2, height + 2);
    const backdrop = scrim.addComponent(Graphics);
    backdrop.fillColor = new Color(79, 66, 55, 92);
    backdrop.rect(-width / 2 - 1, -height / 2 - 1, width + 2, height + 2);
    backdrop.fill();
    const hasDouble = options.doubleAvailable && options.pointsEarned > 0;
    const available = Math.max(360, height - safeTop - safeBottom);
    // The smallest supported safe area (694 - 71 - 46) still fits a 553-high card.
    const cardHeight = Math.min(hasDouble ? 588 : 530, available - 24);
    this.cardWidth = Math.min(344, width - 36);
    const left = (width - this.cardWidth) / 2;
    const top = safeTop + (available - cardHeight) / 2;
    const centerX = width / 2;
    const centerY = top + cardHeight / 2;
    this.round('CardShadowWide', centerX, centerY + 9, this.cardWidth + 2, cardHeight, 32, new
Color(67, 49, 34, 12));
    this.round('CardShadow', centerX, centerY + 4, this.cardWidth, cardHeight, 32, new Color(100,
72, 46, 14));
    this.round('VictoryCard', centerX, centerY, this.cardWidth, cardHeight, 32, C.white);
    // Only the illustration compresses on short screens; body type and hit areas remain readable.
    const heroHeight = Math.max(64, Math.min(150, cardHeight - (hasDouble ? 458 : 400)));
    const heroY = top + 14 + heroHeight / 2;
    if (options.artwork) this.artwork(options.artwork, centerX, heroY, this.cardWidth - 34,
heroHeight);
    else this.fallbackArt(centerX, heroY, heroHeight);
    const titleY = top + 14 + heroHeight + 18;
    this.label('渐变完成', centerX, titleY, this.cardWidth - 40, 34, 25, C.ink, true);
    this.label(`第 ${options.levelNumber} 关 · ${options.levelTitle}`, centerX,
      titleY + 31, this.cardWidth - 40, 22, 12, C.muted);
    const scoreY = titleY + 90;
    const safeScore = Math.max(0, Math.round(Number.isFinite(options.score) ? options.score : 0));
    this.label(String(safeScore), centerX - 8, scoreY, 208, 88, 72, C.ink, true);
    this.label('分', centerX + 82, scoreY + 19, 28, 28, 15, C.muted);
    const pointsY = scoreY + 69;
    const earned = Math.max(0, Math.round(options.pointsEarned));
    this.round('PointsReceipt', centerX, pointsY, this.cardWidth - 48, 42, 15, C.mint);
    this.coin(left + 52, pointsY, 10);
    this.label('积分到账', centerX - 48, pointsY, 94, 28, 13, C.green);
    this.label(`+${earned}`, centerX + 75, pointsY, 106, 32, 21, C.green, true);
    const statsY = pointsY + 53;
    const statsWidth = (this.cardWidth - 58) / 2;
    this.label('用时', centerX - 72, statsY - 12, statsWidth, 20, 12, C.muted);
```

```
    this.label(this.formatTime(options.elapsedMs), centerX - 72, statsY + 11,
      statsWidth, 28, 19, C.ink, true);
    this.label('操作', centerX + 72, statsY - 12, statsWidth, 20, 12, C.muted);
    this.label(`${Math.max(0, Math.round(options.movesSpent))} 次`, centerX + 72, statsY + 11,
      statsWidth, 28, 19, C.ink, true);
    const divider = this.element('StatisticsDivider', centerX, statsY, 1,
30).addComponent(Graphics);
    divider.strokeColor = this.color('#EFE8DE'); divider.lineWidth = 1;
    divider.moveTo(0, -15); divider.lineTo(0, 15); divider.stroke();
    let nextY = statsY + 64;
    if (hasDouble) {
      const adY = statsY + 64;
      this.button('看视频双倍积分', centerX, adY, this.cardWidth - 48, 48,
        C.gold, () => options.onDouble());
      this.play(centerX - 101, adY, C.goldInk);
      this.label('看视频双倍', centerX - 26, adY, 120, 30, 15, C.goldInk, true);
      this.label(`再得 ${earned}`, centerX + 83, adY, 92, 30, 14, C.goldInk, true);
      nextY += 58;
    }
    const nextLabel = options.levelNumber >= 100 ? '返回关卡' : '下一关';
    this.round('PrimaryButtonShadow', centerX, nextY + 4, this.cardWidth - 48, 54, 20, '#DEAD97');
    this.button(nextLabel, centerX, nextY, this.cardWidth - 48, 54,
      C.coral, () => options.onNext());
    this.label(nextLabel, centerX - 5, nextY, this.cardWidth - 104, 32, 18, '#FFFFFF', true);
    this.chevron(centerX + 102, nextY, '#FFFFFF');
    const homeY = nextY + 54;
    this.label('返回首页', centerX, homeY, 180, 26, 13, C.muted);
    this.hit('返回首页', centerX, homeY, 180, 44, () => options.onHome());
  }
  dispose(): void {
    if (this.disposed) return;
    this.disposed = true;
    if (this.node.isValid) { this.node.removeFromParent(); this.node.destroy(); }
  }
  private formatTime(value: number): string {
    const seconds = Math.max(0, Math.floor((Number.isFinite(value) ? value : 0) / 1000));
    return `${Math.floor(seconds / 60)}:${String(seconds % 60).padStart(2, '0')}`;
  }
  private artwork(frame: SpriteFrame, x: number, y: number, maxWidth: number, maxHeight: number):
void {
    const node = this.element('VictoryIllustration', x, y, maxWidth, maxHeight);
    const sprite = node.addComponent(Sprite);
    sprite.sizeMode = Sprite.SizeMode.CUSTOM;
    sprite.spriteFrame = frame;
    const source = frame.originalSize;
    const scale = Math.min(maxWidth / Math.max(1, source.width), maxHeight / Math.max(1,
source.height));
    node.getComponent(UITransform)!.setContentSize(source.width * scale, source.height * scale);
  }
  private fallbackArt(x: number, y: number, height: number): void {
    const scale = Math.min(1, height / 115);
    const g = this.element('CelebrationFallback', x, y, 240, height).addComponent(Graphics);
    g.fillColor = this.color('#FFF2D8'); g.ellipse(0, -7 * scale, 99 * scale, 45 * scale); g.fill();
```

```
    const colors = ['#F4B37F', '#E7B8C9', '#BDC5DE', '#A5D2BD', '#F2D690'];
    colors.forEach((hex, i) => {
      g.fillColor = this.color(hex);
      g.roundRect((i - 2) * 35 * scale - 15 * scale, -14 * scale + Math.abs(i - 2) * 8 * scale,
        30 * scale, 30 * scale, 8 * scale); g.fill();
    });
    this.spark(x - 76 * scale, y - 35 * scale, 7 * scale, '#DEAE59');
    this.spark(x + 82 * scale, y - 20 * scale, 6 * scale, '#D7B0C7');
    this.spark(x + 40 * scale, y - 46 * scale, 5 * scale, '#DEAE59');
  }
  private element(name: string, x: number, y: number, width = 1, height = 1): Node {
    const node = new Node(name); node.layer = Layers.Enum.UI_2D;
    node.addComponent(UITransform).setContentSize(width, height);
    node.setPosition(x - this.options.width / 2, this.options.height / 2 - y);
    this.node.addChild(node); return node;
  }
  private color(value: string | Color): Color { return typeof value === 'string' ? new
Color().fromHEX(value) : value; }
  private round(name: string, x: number, y: number, width: number, height: number,
    radius: number, fill: string | Color): Node {
    const node = this.element(name, x, y, width, height);
    const g = node.addComponent(Graphics); g.fillColor = this.color(fill);
    g.roundRect(-width / 2, -height / 2, width, height, radius); g.fill(); return node;
  }
  private label(value: string, x: number, y: number, width: number, height: number,
    fontSize: number, hex: string, bold = false): void {
    this.options.onText?.(value);
    const label = this.element('VictoryText', x, y, width, height).addComponent(Label);
    label.string = value; label.useSystemFont = true; label.fontFamily = 'sans-serif';
    label.fontSize = fontSize; label.lineHeight = fontSize * 1.2; label.isBold = bold;
    label.color = this.color(hex); label.horizontalAlign = Label.HorizontalAlign.CENTER;
    label.verticalAlign = Label.VerticalAlign.CENTER; label.enableWrapText = false;
    label.overflow = Label.Overflow.SHRINK;
  }
  private hit(label: string, x: number, y: number, width: number, height: number, callback: () =>
void): void {
    const node = this.element(label, x, y, width, height);
    const invoke = () => { if (!this.disposed && this.node.isValid) callback(); };
    node.on(Node.EventType.TOUCH_END, (event: EventTouch) => { event.propagationStopped = true;
invoke(); });
    this.options.registerControl?.(label, invoke, { x, y, width, height });
  }
  private button(label: string, x: number, y: number, width: number, height: number,
    fill: string, callback: () => void): void {
    this.round(label, x, y, width, height, 19, fill);
    this.hit(label, x, y, width, height, callback);
  }
  private coin(x: number, y: number, radius: number): void {
    const g = this.element('PointsCoin', x, y).addComponent(Graphics);
    g.fillColor = this.color('#E8CA80'); g.circle(0, 0, radius); g.fill();
    g.strokeColor = this.color('#FFF5D4'); g.lineWidth = 1.5; g.circle(0, 0, radius - 3);
g.stroke();
    g.moveTo(0, -4); g.lineTo(0, 4); g.stroke();
  }
```

```
  private play(x: number, y: number, hex: string): void {
    const g = this.element('RewardVideoPlay', x, y).addComponent(Graphics); g.fillColor =
this.color(hex);
    g.moveTo(-5, -7); g.lineTo(7, 0); g.lineTo(-5, 7); g.close(); g.fill();
  }
  private chevron(x: number, y: number, hex: string): void {
    const g = this.element('NextChevron', x, y).addComponent(Graphics); g.strokeColor =
this.color(hex); g.lineWidth = 2.3;
    g.moveTo(-3, -5); g.lineTo(3, 0); g.lineTo(-3, 5); g.stroke();
  }
  private spark(x: number, y: number, radius: number, hex: string): void {
    const g = this.element('VictorySpark', x, y).addComponent(Graphics); g.fillColor =
this.color(hex);
    g.moveTo(0, radius); g.lineTo(radius / 3, radius / 3); g.lineTo(radius, 0);
    g.lineTo(radius / 3, -radius / 3); g.lineTo(0, -radius); g.lineTo(-radius / 3, -radius / 3);
    g.lineTo(-radius, 0); g.lineTo(-radius / 3, radius / 3); g.close(); g.fill();
  }
}
// Generated by tools/design/sync-theme.mjs from design-tokens.json. Do not edit.
// Puzzle tile faces always use exact level HEX, never this theme palette.
export const theme = {
  "colors": {
    "background": "#FFF9F0",
    "boardBackground": "#F7F7F2",
    "surface": "#FFFFFF",
    "text": "#54483F",
    "textSecondary": "#75695E",
    "primary": "#E97860",
    "primaryPressed": "#E16E57",
    "onPrimary": "#30221C",
    "mint": "#D9EEE4",
    "lilac": "#E9E2F5",
    "butter": "#FFF0C4",
    "border": "#E9E1D6",
    "track": "#C6D0C9",
    "warning": "#A74630",
    "selection": "#554D72",
    "disabled": "#ECE7DF",
    "scrim": "#54483F"
  },
  "spacing": {
    "unit": 4,
    "xs": 4,
    "sm": 8,
    "md": 12,
    "lg": 16,
    "xl": 24,
    "xxl": 32
  },
  "radii": {
    "tile": 8,
    "button": 26,
    "card": 20,
```

```
    "modal": 28,
    "pill": 99
  },
  "typography": {
    "family": "sans-serif",
    "title": 30,
    "chapter": 18,
    "body": 16,
    "button": 18,
    "caption": 13,
    "moves": 20
  },
  "controls": {
    "minimumTouchSize": 48,
    "primaryHeight": 56,
    "secondaryHeight": 48,
    "iconSize": 24,
    "scrimOpacity": 0.16
  },
  "board": {
    "preferredCellSize": 52,
    "minimumVisibleCellSize": 44,
    "preferredGap": 8,
    "minimumGap": 6,
    "maximumColumns": 6,
    "maximumRows": 6,
    "trayColumns": 6,
    "trayRowsPerPage": 3,
    "trayPageSize": 18
  },
  "motion": {
    "pressMs": 90,
    "placementMs": 160,
    "modalMs": 220,
    "completeMs": 650
  }
} as const;
```